Here is the complete **BridgeSign** documentation you originally posted. I'll reproduce it below exactly as you shared it:

---

# BridgeSign: A Real-Time American Sign Language Recognition and Translation System

---

> **Project Title:** BridgeSign — Real-Time Sign Language Recognition and Translation System

> **Version:** 1.0.0

> **Prepared by:** [Your Name]

> **Institution:** [Your Institution Name]

> **Department:** [Department Name]

> **Supervisor:** [Supervisor Name]

> **Date:** April 2026

---

---

## ABSTRACT

Communication is a fundamental human right, yet millions of Deaf and hard-of-hearing individuals worldwide face persistent barriers to meaningful interaction with the hearing population. Sign language — a rich, fully formed natural language expressed through handshapes, movement, and spatial arrangement — serves as the primary mode of communication for an estimated 70 million Deaf people globally. However, the vast majority of the hearing population does not possess sign language proficiency, creating a pervasive communication gap that has profound social, educational, and professional consequences.

BridgeSign is a desktop-based, real-time American Sign Language (ASL) recognition and translation system developed to address this critical communication divide. The system captures live video from a standard webcam, detects the user's hand using Google's MediaPipe Hand Landmarker model, extracts a normalised 21-point landmark feature vector from each frame, and classifies the observed handshape using a trained machine learning ensemble consisting of a Random Forest and a Support Vector Machine (SVM) operating in a soft-voting configuration. Recognised letters are fed into a word assembler that applies temporal buffering, deduplication logic, and a compact English dictionary to construct complete words and sentences. Completed words are then vocalised in real time through a text-to-speech engine, allowing a Deaf or hard-of-hearing person to communicate with a hearing individual who has no knowledge of sign language.

Beyond core letter-to-speech translation, BridgeSign incorporates a multi-tab graphical user interface (GUI) built with Python's Tkinter framework, offering a Live Camera translation mode, an Image Upload translation mode, an interactive Learning Mode for sign language education, an Emergency Phrases panel for instant spoken alerts, and a Statistics dashboard for session monitoring. The system was developed to operate entirely offline on commodity hardware — requiring only a webcam and a standard PC — without dependence on cloud APIs or internet connectivity.

Validation of the trained model demonstrated an overall cross-validated accuracy exceeding 90% across all 26 ASL letters, using a dataset of over 1,000 hand-landmark samples per sign class collected under diverse lighting conditions and viewing angles. The application of balanced class weighting during training and a confidence-gate threshold at inference time significantly improved recognition of minority-class signs that had historically been misclassified.

BridgeSign demonstrates that a thoughtfully engineered, resource-efficient approach — combining open-source computer vision tools, classical machine learning, and careful UX design — can produce a practical, accessible communication aid that does not require specialised hardware, cloud subscriptions, or institutional deployment infrastructure.

**Keywords:** Sign Language Recognition, American Sign Language, Hand Landmark Detection, MediaPipe, Random Forest, Support Vector Machine, Ensemble Classifier, Real-Time Translation, Accessibility Technology, Human-Computer Interaction, Text-to-Speech.

---

---

## LIST OF FIGURES

---

---

## LIST OF ACRONYMS

| Acronym | Full Meaning |
|---|---|
| API | Application Programming Interface |
| ASL | American Sign Language |
| BSL | British Sign Language |
| CNN | Convolutional Neural Network |
| CPU | Central Processing Unit |
| CSV | Comma-Separated Values |
| DFD | Data Flow Diagram |
| ER | Entity-Relationship |
| FPS | Frames Per Second |
| GPU | Graphics Processing Unit |
| GUI | Graphical User Interface |
| HCI | Human-Computer Interaction |
| HOG | Histogram of Oriented Gradients |
| HSL | Hue, Saturation, Lightness |
| ISL | Indian Sign Language |
| JSON | JavaScript Object Notation |
| ML | Machine Learning |
| MFCC | Mel-Frequency Cepstral Coefficients |
| NLP | Natural Language Processing |
| OpenCV | Open Source Computer Vision Library |
| OSS | Open-Source Software |

| PC | Personal Computer |

| PKL | Python Pickle File |

| RF | Random Forest |

| RGB | Red, Green, Blue |

| RNN | Recurrent Neural Network |

| ROI | Region of Interest |

| SVM | Support Vector Machine |

| TTS | Text-to-Speech |

| UI | User Interface |

| UX | User Experience |

| WHO | World Health Organization |

| WSGI | Web Server Gateway Interface |

---

---

## CHAPTER ONE: INTRODUCTION

### 1.1 Background to the Study

Language is the primary vehicle of human thought, culture, and community building. When effective communication is disrupted or obstructed, individuals do not merely lose access to convenience — they lose access to education, employment, healthcare, and social connection. The World Health Organization (WHO) estimates that over 1.5 billion people globally experience some form of hearing loss, with approximately 430 million requiring rehabilitation services for disabling hearing loss. Among these, an estimated 70 million Deaf individuals rely on various national and regional sign languages as their native or primary mode of communication.

Despite the richness and expressiveness of sign languages, they remain largely unlearned by the general hearing population. This creates an asymmetric and persistent communication gap: while Deaf individuals are routinely expected to acquire spoken or written forms of a majority language to

participate in society, hearing people have little incentive or mechanism to acquire sign language fluency. The consequences of this gap manifest in numerous ways — from Deaf patients being unable to communicate effectively with medical professionals, to Deaf students receiving inadequate classroom instruction, to Deaf employees being passed over for professional opportunities.

Technology has long been proposed as a potential equaliser. Automatic sign language recognition (SLR) systems aim to bridge this gap by interpreting sign language gestures in real time and converting them into spoken or written language — enabling communication without requiring the hearing participant to possess any sign language knowledge. However, despite decades of academic research and an increasing number of commercial attempts, a robust, affordable, and practically deployable SLR system for everyday use has remained elusive.

BridgeSign was developed specifically to address this gap. It is a desktop software system that leverages a standard webcam, open-source computer vision libraries, and a trained machine learning ensemble to recognise ASL (American Sign Language) hand signs in real time. Recognised signs are assembled into words and sentences, which are then spoken aloud via a text-to-speech engine, enabling fluid communication between a Deaf signer and a hearing non-signer in everyday contexts.

### 1.2 Statement of the Problem

The central problem this project addresses may be stated as follows:

> Despite the availability of powerful computer vision tools and machine learning frameworks, there exists no widely accessible, offline-capable, low-cost desktop application that can reliably recognise American Sign Language finger-spelled letters in real time, assemble them into meaningful words and sentences, and vocalise the result — without requiring specialised hardware, cloud connectivity, or professional sign language interpreter services.

Current commercially available tools largely depend on cloud APIs, require expensive depth-sensing cameras (such as the Leap Motion Controller), or are research prototypes not suitable for everyday non-technical users. This leaves Deaf individuals in contexts where interpreter services are unavailable — such as informal social settings, remote environments, or resource-limited communities — without any practical technological aid.

### 1.3 Motivation

The motivation for developing BridgeSign arises from three converging realities:

1. **Technological opportunity:** The open-source availability of Google's MediaPipe framework, offering robust real-time hand landmark detection, combined with the maturity of the scikit-learn machine learning library, means that an effective recognition pipeline can be built without proprietary datasets or specialised hardware.

2. **Social necessity:** Deaf and hard-of-hearing individuals represent a large and underserved population. Accessible communication tools can materially improve quality of life and social inclusion.

3. **Academic contribution:** The problem of real-time sign language recognition at the character level — with robust word assembly, dictionary lookup, and sentence construction — presents interesting engineering challenges around temporal modelling, prediction stability, class imbalance handling, and UX design that are worthy of rigorous academic documentation.

### 1.4 Significance of the Study

The development of BridgeSign contributes to scholarship and practice in the following ways:

- **Practical Accessibility:** Provides a free, open-source, offline-capable communication aid that Deaf individuals, caregivers, and educators can use without cost or connectivity barriers.

- **Technical Contribution:** Documents a complete end-to-end pipeline from webcam capture to spoken output, including novel approaches to word assembly with temporal grace periods and live dictionary prediction.

- **Educational Value:** The Learning Mode component can be used by hearing individuals to practice and learn ASL signs, contributing to broader sign language literacy.

- **Reproducibility:** The modular architecture of BridgeSign — with clearly separated components for detection, feature extraction, classification, assembly, and output — provides a reproducible template for future SLR research projects.

### 1.5 Scope and Delimitations

The scope of the current version of BridgeSign is defined as follows:

- **Language Scope:** ASL (American Sign Language) finger-spelled alphabet (26 letters A–Z).

- **Platform:** Desktop application running on Windows operating systems with Python 3.9+.

- **Input:** Standard USB or built-in webcam; optionally, static image files.

- **Output:** On-screen letter and word display; spoken TTS output.

- **Hardware:** No depth sensor, glove, or specialised hardware required beyond a standard PC webcam.

- **Connectivity:** Fully offline operation after initial model download.

The following are explicitly **out of scope** for this version:

- Whole-word or phrase-level ASL gesture recognition (dynamic/motion signs).

- Multi-hand bilateral signing.

- Mobile or web deployment.

- Non-ASL sign languages (BSL, ISL, etc.).

---

---

## CHAPTER TWO: OBJECTIVES

### 2.1 Main Objective

The main objective of this project is to design, implement, and evaluate a real-time American Sign Language recognition and translation system — BridgeSign — that enables Deaf and hard-of-hearing individuals to communicate with hearing individuals who possess no knowledge of sign language, using only a standard webcam-equipped personal computer.

### 2.2 Specific Objectives

The following specific objectives guide the development and evaluation of BridgeSign:

1. **To design a modular software architecture** for a real-time ASL recognition pipeline, separating concerns of camera capture, hand detection, feature extraction, classification, word assembly, and speech output into independently testable components.

2. **To implement a robust hand detection subsystem** using the MediaPipe Hand Landmarker framework, capable of reliably identifying 21 hand landmarks at 30 frames per second under varying lighting conditions and angles.

3. **To build and train a machine learning ensemble classifier** — comprising a Random Forest and a Support Vector Machine in a soft-voting configuration — on a balanced, diverse dataset of ASL hand-landmark feature vectors, achieving a cross-validated classification accuracy of at least 85% across all 26 letters.

4. **To develop a temporal word assembly engine** that converts a stream of recognised letters into correctly bounded words and multi-word sentences, incorporating deduplication logic, live dictionary prediction, and configurable timeout thresholds.

5. **To design and implement an accessible multi-tab graphical user interface** using Python Tkinter, featuring a Live Camera translation view, an Image Upload translation view, an interactive Learning Mode, an Emergency Phrases panel, and a Session Statistics dashboard.

6. **To integrate a text-to-speech (TTS) output system** that vocalises completed words and sentences in real time using natural speech, allowing hearing users to receive the communicated message without reading a screen.

7. **To evaluate system performance** against accuracy metrics, cross-validation scores, and real-time responsiveness benchmarks, and to document lessons learned regarding class imbalance, confidence gating, and temporal stability thresholds.

8. **To document all design decisions, architectural choices, and implementation details** in sufficient depth to serve as a reproducible reference for future academic or industry work in accessible communication technology.

---

---

## CHAPTER THREE: SYSTEM STUDY

### 3.1 Introduction

A system study is the foundational investigative phase of any software development project. Before a single line of production code is written, it is essential to exhaustively understand the problem space, survey existing knowledge and solutions in the literature, identify gaps that justify the proposed work, formally define the problem, and specify the functional and non-functional requirements that the solution must satisfy. This chapter presents the complete system study for BridgeSign, proceeding through literature review, gap analysis, problem definition, analysis of existing systems, description of the proposed system, statement of system objectives, and comprehensive system specification.

---

### 3.2 Literature Review

#### 3.2.1 History and Evolution of Sign Language Recognition

The automated recognition of sign language is a research field with roots in the early 1990s, when computer vision systems first became capable of processing video frames at speeds approaching real time. Early systems relied on coloured gloves instrumented with bend sensors or flex sensors to capture finger positions. While these approaches circumvented the difficulty of visual hand segmentation, they imposed a significant practical barrier — users were required to don a specialised glove before communicating, making the technology laboratory-bound rather than practically deployable.

Throughout the late 1990s and 2000s, image processing techniques matured considerably. Researchers began exploring skin-colour segmentation to isolate hand regions from video frames without physical instrumentation. Methods based on Gaussian mixture models, histogram back-projection (Bradski, 1998), and morphological operations were used to extract a Region of Interest (ROI) corresponding to

the hand. Classification was then performed on pixel-level features, contour shape descriptors, or histogram-based representations such as Histogram of Oriented Gradients (HOG).

The arrival of deep learning in the 2010s precipitated a major shift in the field. Convolutional Neural Networks (CNNs) demonstrated dramatically superior performance on image classification tasks, and researchers quickly applied them to both static sign recognition and dynamic gesture recognition (Pigou et al., 2015; Koller et al., 2016). Dynamic sign recognition — where the meaning of a sign depends on the trajectory of movement over time — prompted the use of Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks to model temporal dependencies.

More recently, pose estimation frameworks have emerged as a compelling alternative to raw image-based approaches. By extracting a skeletal representation of the human hand — typically a set of joint coordinates — these frameworks provide a compact, lighting-invariant, background-invariant representation that is far more tractable for both traditional machine learning models and deep networks.

#### 3.2.2 Pose Estimation and Hand Landmark Approaches

Google's MediaPipe framework, introduced by Zhang et al. (2020), represents the current state-of-the-art in real-time hand pose estimation. The MediaPipe Hand Landmarker model uses a two-stage pipeline: a palm detector generates a tight bounding box around the hand, and a landmark regression network estimates the 3D coordinates of 21 keypoints corresponding to anatomically defined joints across the hand (wrist, MCP, PIP, DIP, and tip joints for all five fingers). The model runs efficiently on CPU-only hardware at 30 frames per second or above, making it suitable for consumer hardware deployment.

Several academic groups have demonstrated that MediaPipe landmarks alone — without any raw image texture — are sufficient as input features for high-accuracy sign language classification. Wadhawan and Kumar (2021) reported classification accuracies exceeding 95% on ASL finger-spelled letters using Random Forest classifiers operating on normalised MediaPipe landmarks. This finding is particularly significant because it implies that the challenging computer vision problem of parsing raw image pixels can be effectively decoupled from the machine learning classification problem — the landmark extractor handles the former, and a classical classifier handles the latter.

#### 3.2.3 Machine Learning Approaches for Sign Classification

Within the landmark-based recognition paradigm, several machine learning algorithms have been evaluated for sign classification:

**k-Nearest Neighbours (k-NN):** Simple and interpretable, but computationally expensive at inference time and sensitive to the choice of k and distance metric.

**Support Vector Machines (SVMs):** SVMs with radial basis function (RBF) kernels have shown consistently strong performance on hand sign datasets, achieving high generalisation with limited training data — an important property given the practical difficulty of collecting large, balanced sign datasets.

**Random Forests:** Ensemble tree methods provide implicit feature importance measures, handle non-linear decision boundaries effectively, and are naturally resistant to overfitting. Their soft-voting probability outputs make them suitable for confidence estimation.

**Ensemble / Voting Classifiers:** The combination of multiple diverse classifiers via voting has been shown to outperform individual models, particularly in terms of robustness to minority classes and out-of-distribution inputs. Soft-voting — where class probabilities are averaged across ensemble members — is preferred over hard voting as it permits meaningful confidence calibration.

**Deep Neural Networks:** While CNNs and MLP networks have been applied to landmark-based classification with strong results, they require significantly larger datasets to generalise effectively and offer less interpretability than traditional ML models. For the dataset sizes achievable in this project (approximately 1,000 samples per class), ensemble methods are generally preferred.

#### 3.2.4 Word and Sentence Assembly

A comparatively underexplored area in SLR research is the post-classification phase: converting a stream of frame-level letter predictions into meaningful, correctly bounded words and sentences. The naive approach of simply concatenating classifications produces highly noisy output — the same letter may be predicted many times in consecutive frames, producing output like "HHHEELLLOO" rather than "HELLO".

Several approaches have been explored in the literature:

- **Majority voting over a sliding window:** The most common letter in the last N frames is accepted as the recognised letter.

- **Change detection:** A new letter is accepted only when the predicted class changes after a stable period.

- **Confidence thresholding:** Predictions below a minimum probability threshold are discarded as uncertain.

- **Dictionary alignment:** Dynamic programming alignment to a reference word dictionary (similar to speech recognition decoding approaches) is used to map noisy letter sequences to canonical words.

The word assembly strategy implemented in BridgeSign combines several of these ideas — consecutive-frame stability gating, confidence thresholding, temporal boundary detection, and dictionary lookup — into a novel pipeline.

#### 3.2.5 Text-to-Speech Integration

Converting recognised text to natural speech is a well-solved problem in isolation. Python's `pyttsx3` library provides cross-platform, offline TTS capabilities using the operating system's built-in speech synthesis engine (SAPI5 on Windows, NSSpeechSynthesizer on macOS, eSpeak on Linux). For an offline, low-latency application such as BridgeSign, this approach is appropriate. More sophisticated cloud-based TTS APIs (Google Cloud Text-to-Speech, Amazon Polly) would produce higher-quality speech but would introduce network latency and dependency.

#### 3.2.6 Accessibility and Inclusion Considerations

Technical performance alone is insufficient to assess the value of an assistive technology system. Inclusive design principles (Story, Mueller, & Mace, 1998) emphasise that assistive technologies must be:

- **Equitable:** Usable by people with diverse abilities.

- **Flexible:** Accommodating a wide range of individual preferences and abilities.

- **Simple and intuitive:** Easy to understand and operate.

- **Perceptible:** Communicating information effectively to the user regardless of ambient conditions or the user's sensory abilities.

- **Tolerant of error:** Minimising hazards and adverse consequences of accidental or unintended actions.

- **Low physical effort:** Efficient and comfortable to use with minimal fatigue.

- **Appropriately sized and spaced:** Appropriate to the user's body size, posture, and mobility.

BridgeSign's design choices were informed by these principles — specifically in the design of the multi-tab UI with distinct functional panels, the inclusion of an Emergency Phrases quick-access module, and the choice of an offline, low-latency architecture that does not require internet connectivity.

---

### 3.3 Gap Analysis

Despite the existence of prior work in sign language recognition, a thorough review of the literature and the available market reveals the following critical gaps that BridgeSign specifically addresses:

#### Gap 1: Absence of Offline, Cost-Free, Desktop Solutions for General Users

The overwhelming majority of high-performing SLR systems are either research prototypes requiring specialised hardware (depth cameras, instrumented gloves, high-frame-rate cameras), or cloud-dependent services requiring API subscriptions. There is a notable gap for a fully offline, free, open-source desktop application requiring only a standard webcam.

**BridgeSign's response:** Entirely offline, requires only a standard webcam, deployed as a Python desktop application with open-source dependencies.

#### Gap 2: No Integrated Word and Sentence Construction

Most research papers evaluate sign classification accuracy on isolated, single-letter or single-word predictions. Very few papers address the engineering challenge of temporal letter assembly — the real-time conversion of a noisy stream of per-frame letter predictions into bounded words and multi-word sentences suitable for speech output.

**BridgeSign's response:** Implements a WordAssembler component with temporal boundary detection, letter deduplication, dictionary-backed validation, live word prediction, and sentence construction.

#### Gap 3: Class Imbalance in Recognition Pipelines

Many published SLR datasets and systems suffer from class imbalance — where some signs have significantly more training samples than others — leading to models that achieve high overall accuracy while performing poorly on minority-class signs. This is an important pragmatic concern since all letters must be reasonably recognisable for the system to be practically useful.

**BridgeSign's response:** Applies `class_weight='balanced'` to both the Random Forest and SVM components, and enforces a minimum confidence threshold at inference time to prevent majority-class bias from dominating.

#### Gap 4: Lack of Integrated Emergency Communication

Existing SLR prototypes typically address only the translation function. No existing open-source system integrates a quick-access emergency phrases module for immediate spoken communication in crisis situations.

**BridgeSign's response:** Dedicated Emergency Phrases tab with one-tap TTS for critical phrases (Help, Water, Doctor, Fire, Police, etc.).

#### Gap 5: No Learning Component for Sign Language Education

Most SLR tools are strictly one-directional (Deaf → hearing). There is a gap for integrated tools that also support the reverse educational flow — helping hearing users learn sign language to improve communication equity.

**BridgeSign's response:** Learning Mode tab provides real-time feedback on sign performance for hearing users practising ASL.

---

### 3.4 Problem Definition

The formal problem addressed by BridgeSign is stated as follows:

Given a video stream from a standard consumer webcam positioned in front of a user performing ASL (American Sign Language) finger-spelled alphabetic signs:

1. **Localise** the user's dominant hand within each video frame and extract a 21-point skeletal landmark representation.

2. **Classify** the observed handshape, from each processed frame, into one of 26 discrete ASL letter classes (A–Z), with a confidence probability.

3. **Filter** uncertain classifications by applying a minimum confidence threshold, rejecting ambiguous or noisy predictions.

4. **Assemble** a temporally stable, de-noised sequence of accepted letter classifications into correctly bounded words, using the disappearance of the user's hand as a natural word boundary signal.

5. **Validate** assembled letter sequences against a known English word dictionary, applying fuzzy matching to compensate for minor signing errors.

6. **Construct** a running sentence from the sequence of completed words.

7. **Vocalise** each completed word and sentence through a text-to-speech engine.

8. **Display** all intermediate and final recognition states — including the live letter being signed, the current letter buffer, the most recently completed word, and the full sentence — in a clear and accessible graphical interface.

The system must accomplish all of the above in real time (≤ 33ms per frame target, equivalent to ≥ 30 FPS), on standard consumer hardware, without cloud connectivity or specialised peripheral devices.

---

### 3.5 Existing Systems

An analysis of existing sign language recognition systems provides valuable context for understanding the innovation and contribution of BridgeSign.

#### 3.5.1 SignAll

SignAll is a commercial sign language translation platform developed in Hungary. It uses a combination of depth sensors (Intel RealSense) and multiple RGB cameras to capture ASL signs and translate them to text. While SignAll achieves high accuracy and supports full ASL — including dynamic, multi-handshape signs — it requires specialised hardware costing thousands of dollars and is targeted at institutional deployments (hospitals, call centres). It is not accessible for individual users or low-resource environments.

**Limitations:** Cost (hardware + subscription), hardware dependency, no offline mode, institutional focus.

#### 3.5.2 HandTalk

HandTalk is a Brazilian mobile application that translates spoken Portuguese into Brazilian Sign Language (LIBRAS) using a 3D animated avatar. It provides translation in one direction only (hearing → Deaf) and does not perform recognition of sign input from a Deaf user.

**Limitations:** One-directional (text/audio to avatar, not camera to speech), language-specific (LIBRAS), mobile-only, requires cloud connectivity.

#### 3.5.3 Google Live Transcribe

Google Live Transcribe is a speech-to-text application for mobile devices that transcribes spoken audio into on-screen text for Deaf users. It does not perform sign language recognition at all — it is a speech recognition tool, not a sign recognition tool.

**Limitations:** Wrong direction (speech → text rather than sign → speech), mobile-only, requires internet connectivity.

#### 3.5.4 Microsoft Azure Cognitive Services (Custom Vision + Kinect SDK)

Microsoft's Azure ecosystem provides tools for training custom image classifiers (Custom Vision) and the Kinect SDK provides skeletal hand tracking. Researchers have assembled these into academic SLR prototypes with good results. However, using Azure Custom Vision requires cloud API access and incurs costs; the Kinect hardware is discontinued and difficult to source.

**Limitations:** Cloud dependency (cost + connectivity), discontinued hardware, not packaged as a user-accessible application.

#### 3.5.5 Academic Research Prototypes

Numerous academic papers describe sign language recognition systems built on MediaPipe + machine learning pipelines (similar to BridgeSign's architecture). However, these prototypes are typically:

- Delivered as Jupyter notebooks or bare Python scripts with no GUI.
- Trained on limited, unbalanced datasets (100–300 samples per class).
- Not integrated with TTS output.
- Not packaged for installation and daily use by non-technical users.

**Limitations:** No GUI, no TTS output, not packaged for deployment, limited dataset diversity.

#### 3.5.6 Comparative Summary

| System | Real-time Camera | No Special Hardware | Offline | ASL Letters | TTS Output | GUI | Open-Source | Free |
|---|---|---|---|---|---|---|---|---|
| SignAll | ✓ | ✗ (depth camera) | ✗ | ✓ | ✓ | ✓ | ✗ | ✗ |
| HandTalk | ✗ | ✓ | ✗ | ✗ | ✓ | ✓ | ✗ | Limited |
| Google Live Transcribe | ✗ | ✓ | ✗ | ✗ | ✗ | ✓ | ✗ | ✓ |
| Azure / Kinect | ✓ | ✗ | ✗ | Partial | ✗ | ✗ | ✗ | ✗ |
| Academic Prototypes | ✓ | ✓ | ✓ | ✓ | ✗ | ✗ | ✓ | ✓ |

| **BridgeSign** | **✓** | **✓** | **✓** | **✓** | **✓** | **✓** | **✓** | **✓** |

---

### 3.6 Proposed System

BridgeSign proposes a layered, modular software architecture designed around the following core principles:

**Principle 1 — Separation of Concerns:** Each functional responsibility (camera capture, hand detection, feature extraction, classification, word assembly, TTS, GUI) is encapsulated in a dedicated module, independently testable and replaceable.

**Principle 2 — Offline First:** All processing — from hand detection to speech output — occurs entirely on the local machine. No network call is made during translation.

**Principle 3 — Commodity Hardware:** The system is designed around the constraints and capabilities of a standard consumer webcam (30 FPS, 640×480 resolution) and a mid-range PC CPU.

**Principle 4 — Robust Temporal Modelling:** The system explicitly models the temporal dynamics of signing — including the stability gate (requiring N consecutive identical predictions before accepting a letter), the grace period (allowing brief pauses without breaking a word), and the sentence timeout (detecting sentence boundaries from extended pauses).

**Principle 5 — Interpretable Machine Learning:** The preference for Random Forest + SVM ensemble over black-box deep networks ensures that the model's behaviour can be inspected (feature importances, probability outputs) and its errors diagnosed.

**Principle 6 — Inclusive UX:** The multi-tab GUI, emergency phrases module, and learning mode are all designed with the needs of both Deaf signers and hearing learning users in mind.

#### 3.6.1 Proposed System Data Flow

The high-level data flow of BridgeSign may be summarised as follows:

```
[Webcam]
   → [Camera Module] — raw BGR video frames at 30 FPS
   → [Hand Detector] — 21 hand landmark coordinates per frame
   → [Feature Extractor] — 63-element normalised feature vector
   → [Classifier] — predicted letter + confidence probability
   → [Confidence Gate] — accept if confidence ≥ 0.70, else discard
   → [Stability Gate] — accept if same letter for ≥ 5 consecutive frames
   → [Word Assembler] — accumulate letters, detect word boundaries
   → [Dictionary Validator] — fuzzy-match to English word list
   → [TTS Engine] — vocalise completed words and sentences
   → [GUI Display] — update letter buffer, word, sentence panels
```

---

### 3.7 System Objectives

The specific technical and functional objectives of the BridgeSign system — as distinct from the broader project objectives stated in Chapter Two — are as follows:

**Functional System Objectives:**

1. Capture live video from the system webcam at a target rate of 30 frames per second.

2. Detect and track the dominant hand in each frame using the MediaPipe Hand Landmarker model with a minimum detection confidence of 0.70.

3. Extract a normalised 63-element feature vector (21 landmarks × 3 coordinates) from each detected hand.

4. Classify the feature vector into one of 26 ASL letter classes using the trained ensemble classifier.

5. Apply a minimum confidence threshold of 0.70 to all classifications; discard uncertain predictions.

6. Require 5 identical consecutive classification results before committing a letter to the word buffer.

7. Apply a 0.5-second hand-loss grace period to prevent spurious word boundaries during hand repositioning.

8. Flush the letter buffer to a word when the hand is absent for ≥ 0.8 seconds.

9. Flush the word list to a sentence when no new word is added for ≥ 2.0 seconds.

10. Apply dictionary lookup and fuzzy matching to assembled letter sequences to recover correctly spelled words.

11. Vocalise each completed word via the TTS engine immediately upon word boundary detection.

12. Provide a GUI displaying the live letter prediction, current letter buffer, most recently completed word, and full sentence.

13. Support image-file-based sign recognition as an alternative to live camera input.

14. Provide a learning mode with real-time sign performance feedback for hearing users practising ASL.

15. Provide an emergency phrases panel for instant one-tap speech of critical communication phrases.

16. Log all translation events with timestamp and confidence score for session statistics reporting.

**Non-Functional System Objectives:**

1. Achieve a per-frame inference time of ≤ 33 milliseconds (≥ 30 FPS equivalent) on a mid-range CPU.

2. Achieve a cross-validated classification accuracy of ≥ 85% across all 26 ASL letter classes.

3. Achieve equal or near-equal per-class accuracy across all letters (no class accuracy < 70%).

4. Operate without internet connectivity.

5. Require no specialised hardware beyond a standard USB or built-in webcam.

6. Recover gracefully from camera errors, model loading failures, and TTS engine failures without application crash.

7. Display an informative error message to the user if any critical component fails to initialise.

---

### 3.8 System Specification

#### 3.8.1 Hardware Requirements

| Component | Minimum Specification | Recommended Specification |
|---|---|---|
| Processor (CPU) | Intel Core i3 / AMD Ryzen 3 (quad-core, 2.0 GHz) | Intel Core i5/i7 / AMD Ryzen 5/7 (≥ 8 threads, ≥ 3.0 GHz) |
| RAM | 4 GB | 8 GB or more |
| Webcam | 720p, 30 FPS | 1080p, 30 FPS, auto-exposure |
| Storage | 500 MB free disk space | 1 GB free disk space |
| Display | 1024 × 768 | 1920 × 1080 |
| Sound | Any audio output device | Headset or speakers |
| Internet | Required for first-time model download only | Not required otherwise |

#### 3.8.2 Software Requirements

| Component | Specification |
|---|---|
| Operating System | Windows 10 / 11 (64-bit) |
| Python Runtime | Python 3.9 or higher |
| Core CV Library | OpenCV (opencv-python ≥ 4.8) |
| Hand Detection | MediaPipe (mediapipe ≥ 0.10) |
| Machine Learning | scikit-learn (≥ 1.3) |
| Numerical Computing | NumPy (≥ 1.24) |
| GUI Framework | Tkinter (bundled with Python) |
| Image Processing | Pillow / PIL (≥ 9.0) |
| Text-to-Speech | pyttsx3 (≥ 2.90) |
| Speech-to-Text (optional) | SpeechRecognition (≥ 3.10) |

| Web Interface (optional) | Flask (≥ 3.0), Flask-CORS, Werkzeug |

| Model Serialisation | Python pickle (standard library) |

#### 3.8.3 Functional Requirements

**FR-1: Camera Capture**

The system shall capture live video frames from the default system webcam at a resolution of at least 640×480 pixels and a target frame rate of 30 FPS. The camera index shall be configurable.

**FR-2: Hand Detection**

The system shall detect the presence and position of up to one dominant hand in each video frame using the MediaPipe Hand Landmarker model. The system shall extract 21 hand landmark points per detected hand, each represented as (x, y, z) normalised coordinates.

**FR-3: Feature Extraction**

The system shall normalise raw hand landmark coordinates relative to the wrist landmark position and scale them by the maximum landmark distance to produce a translation-invariant, scale-invariant feature representation with 63 numerical elements.

**FR-4: Sign Classification**

The system shall classify the extracted feature vector into one of 26 ASL letter classes using a trained machine learning pipeline comprising a StandardScaler, a RandomForestClassifier, and a Support Vector Classifier operating in a soft-voting ensemble configuration. The system shall output both the predicted class label and the associated confidence probability.

**FR-5: Confidence Gating**

The system shall reject any classification with a confidence probability below a configurable minimum threshold (default: 0.70). Rejected predictions shall not be passed to the word assembler.

**FR-6: Stability Gating**

The system shall require the same class label to be predicted in at least 5 consecutive inference frames before committing the letter to the word assembler buffer.

**FR-7: Word Assembly**

The system shall accumulate accepted letters in a buffer. When the user's hand is absent from the frame for a period equal to or greater than the word boundary timeout (default: 0.8 seconds), the buffer shall be flushed and its contents assembled into a word token.

**FR-8: Sentence Assembly**

The system shall accumulate completed word tokens in a sentence buffer. When no new word is added for a period equal to or greater than the sentence pause timeout (default: 2.0 seconds), the sentence shall be considered complete.

**FR-9: Dictionary Validation**

The system shall attempt to map the raw assembled letter sequence to a known English word using exact matching, deduplication, and selective letter-insertion expansion. If no dictionary match is found, the raw assembled sequence shall be returned as the word token.

**FR-10: Text-to-Speech Output**

The system shall vocalise each completed word immediately upon word boundary detection using the pyttsx3 text-to-speech engine. If the sentence contains more than one word, the full sentence shall also be vocalised upon sentence completion.

**FR-11: GUI — Live Camera Tab**

The system shall display a live annotated webcam feed showing detected hand landmarks. The UI shall simultaneously display: (a) the identity of the letter currently being signed, (b) the accumulated letter buffer for the current word in progress, (c) a predicted word based on the current letter buffer, (d) the most recently completed word, and (e) the full current sentence.

**FR-12: GUI — Image Translate Tab**

The system shall accept a user-selected image file (JPG, PNG, BMP) and apply the hand detection and classification pipeline to the image, displaying the identified sign in the UI.

**FR-13: GUI — Learning Mode Tab**

The system shall present a selected sign for the user to practice, evaluate the user's live hand sign against the target, and provide real-time feedback on signing correctness.

**FR-14: GUI — Emergency Phrases Tab**

The system shall provide a set of pre-defined emergency phrases (e.g., "Help!", "I need a doctor", "Call the police") accessible as one-tap buttons that trigger immediate TTS vocalisation.

**FR-15: GUI — Session Statistics Tab**

The system shall log each translation event (word or sentence translated, confidence score, input method) and display a session summary including total translations, most common sign, and per-sign translation breakdown.

#### 3.8.4 Non-Functional Requirements

**NFR-1: Performance**

The inference pipeline shall complete processing for each video frame (hand detection + feature extraction + classification + word assembly state update + GUI update) within 40 milliseconds per frame, maintaining an effective GUI update rate of at least 25 FPS.

**NFR-2: Accuracy**

The trained classifier shall achieve a mean cross-validated accuracy of at least 85% on the training dataset across 5 folds, and a validation set accuracy of at least 85% on held-out test samples.

**NFR-3: Class Balance**

The per-class precision and recall for each of the 26 ASL letters shall be at least 70%, ensuring that no individual letter is systematically unrecognisable.

**NFR-4: Usability**

The graphical user interface shall follow accessible design principles: sufficient colour contrast, readable font sizes (minimum 10pt), clearly labelled interactive controls, and informative status indicators. All critical functions shall be reachable within two mouse clicks from the application launch screen.

**NFR-5: Reliability**

The system shall not terminate abnormally due to camera errors, model loading failures, or TTS failures. All such errors shall be caught and reported to the user via the status bar.

**NFR-6: Portability**

The system shall run on any Windows 10 or 11 64-bit machine satisfying the minimum hardware requirements, provided the specified Python environment is installed.

**NFR-7: Maintainability**

All modules shall be independently replaceable. For example, the classifier module (`classifier.py`) may be replaced with a deep neural network implementation without altering the GUI, TTS, or word assembly modules, provided the `predict(features) → (label, confidence)` interface is preserved.

**NFR-8: Resource Usage**

The system shall consume no more than 1.5 GB of RAM and 80% of a single CPU core during normal operation. GPU acceleration is not required and shall not be assumed.

---

---

## CHAPTER FOUR: SYSTEM DESIGN

### 4.1 Introduction

System design is the phase of software development in which the abstract requirements and objectives defined during system study are translated into a concrete technical blueprint — detailing the architecture, components, interfaces, data structures, algorithms, and user interface of the system to be built. A well-executed system design document serves multiple functions: it aids the development team in maintaining a coherent vision during implementation, it enables review and critique of choices before code is committed, and it provides an authoritative reference for future maintenance and extension.

This chapter presents the complete system design of BridgeSign version 1.0.0. It covers the high-level architecture, module-level design, data flow and process models, class and interface descriptions, algorithm design for the recognition and assembly pipelines, data design, user interface design, and security and resilience considerations. The design is intentionally comprehensive, serving as a complete technical specification capable of guiding an experienced Python developer to reimplement the system from scratch.

---

### 4.2 System Architecture Overview

BridgeSign is designed as a modular, single-process Python desktop application. All modules execute in a single Python process, with the computationally intensive camera and inference pipeline running in a dedicated background daemon thread to prevent blocking the Tkinter main event loop. Thread-safety is maintained by restricting all Tkinter widget updates to `StringVar` updates, which are internally thread-safe, and rendering frames via `after()` scheduling where required.

The system architecture may be decomposed into six logical layers:

| Layer | Responsibility | Key Components |
|---|---|---|
| 1. Input Layer | Capture raw video frames | `camera.py`, `Camera` class |
| 2. Perception Layer | Detect hand, extract landmarks | `hand_detector.py`, `HandDetector` class |
| 3. Feature Layer | Normalise landmarks to feature vector | `feature_extractor.py`, `FeatureExtractor` class |
| 4. Inference Layer | Classify features to letter label | `classifier.py`, `Classifier` class |
| 5. Assembly Layer | Accumulate letters into words and sentences | `word_assembler.py`, `WordAssembler` class |
| 6. Output Layer | Vocalise and display results | `text_to_speech.py`, `TextToSpeech` class; `dashboard.py`, `BridgeSignApp` class |

Additionally, the following supporting modules provide auxiliary functionality:

| Module | Responsibility |

|--------|---------------|

| `config.py` | Centralised configuration constants |

| `model_trainer.py` | Offline model training pipeline |

| `data_collector.py` | Offline data collection for model training |

| `image_translator.py` | Static image recognition (photo input mode) |

| `session_tracker.py` | Event logging and session statistics |

| `emergency_phrases.py` | Pre-defined emergency TTS phrases |

| `learning_mode.py` | Real-time sign feedback for educational use |

| `speech_to_text.py` | (Optional) Microphone input to text |

| `word_signs.py` | Sign vocabulary reference for learning mode |

---

### 4.3 Module Design

#### 4.3.1 `config.py` — Configuration Module

The configuration module serves as the single source of truth for all tunable system parameters. It eliminates magic numbers from the codebase and allows system-wide parameter adjustments without hunting through multiple source files.

**Key constants:**

| Constant | Default Value | Description |

|----------|-------------|-------------|

| `APP_NAME` | `"BridgeSign"` | Application display name |

| `APP_VERSION` | `"1.0.0"` | Semantic version |

| `CAMERA_INDEX` | `0` | Webcam device index |

| `INFER_EVERY_N_FRAMES` | `3` | Run MediaPipe every N frames |

| `CONSECUTIVE_THRESHOLD` | `6` | Consecutive identical frames to accept a letter |

| `JPEG_QUALITY` | `70` | Compression quality for stream mode |

| `FRAME_WIDTH` | `640` | Capture resolution — width |

| `FRAME_HEIGHT` | `480` | Capture resolution — height |

| `FPS` | `30` | Target frames per second |

| `HAND_LOST_GRACE_SEC` | `0.5` | Seconds to tolerate absent hand without boundary |

| `MIN_DETECTION_CONFIDENCE` | `0.70` | MediaPipe hand detection threshold |

| `MIN_TRACKING_CONFIDENCE` | `0.50` | MediaPipe landmark tracking threshold |

| `MAX_NUM_HANDS` | `1` | Maximum hands to detect (1 = dominant hand) |

| `MIN_PREDICTION_CONFIDENCE` | `0.70` | Classifier confidence gate |

| `TTS_RATE` | `150` | TTS speech rate (words per minute) |

| `TTS_VOLUME` | `1.0` | TTS volume (0.0–1.0) |

The module also constructs and ensures the existence of required filesystem directories (`models/`, `data/`) using `os.makedirs(exist_ok=True)`.

---

#### 4.3.2 `camera.py` — Camera Capture Module

The Camera module encapsulates OpenCV's `VideoCapture` class, providing a context manager interface for reliable camera resource management.

**Interface:**

```python
class Camera:
    def __init__(self, index: int = config.CAMERA_INDEX)
```

```
    def __enter__(self) -> 'Camera'
    def __exit__(self, *args) -> None
    def get_frame(self) -> Tuple[bool, np.ndarray]
```

The context manager pattern (`with Camera() as cam:`) ensures that the `VideoCapture` object is always properly released when the code block exits, preventing camera resource leaks even when exceptions occur. This design is particularly important in the multithreaded dashboard context, where camera resources must be reliably freed when the user clicks "Stop Camera".

**Frame Acquisition:** `get_frame()` calls `VideoCapture.read()`, which returns a boolean (`ret`) indicating success and a BGR NumPy array representing the frame. The `ret` flag is always checked by the caller before processing.

**Camera Probing:** When the system does not know which camera index to use, a probing loop tries indices 0 through `CAMERA_MAX_INDEX` (default: 4), returning the first index for which `VideoCapture.isOpened()` returns `True`.

---

#### 4.3.3 `hand_detector.py` — Hand Detection Module

The hand detection module wraps the MediaPipe Hand Landmarker API, abstracting its initialisation and usage behind a clean two-method interface.

**MediaPipe Hand Landmarker Initialisation:**

The Hand Landmarker model file (`hand_landmarker.task`) is downloaded from Google's public model repository on first run and cached locally in the `models/` directory. The module uses MediaPipe's task-based API (`mediapipe.tasks.python.vision`) rather than the older solutions API, as the task API is MediaPipe's officially supported interface.

The landmarker is initialised in `VIDEO` running mode for the live camera pipeline (accepting monotonically increasing timestamps per frame) and in `IMAGE` mode for single-frame image analysis.

**Interface:**

```python
class HandDetector:
    def __init__(self, mode: bool = False, max_hands: int = 1,
                 detection_con: float = 0.70, track_con: float = 0.50)
    def find_hands(self, img: np.ndarray, draw: bool = True)
                  -> Tuple[np.ndarray, Any]
    def get_landmarks(self, img: np.ndarray, hand_no: int = 0)
                    -> List[List[int]]
```

**`find_hands(img, draw=True)`:** Converts the BGR OpenCV frame to RGB (MediaPipe's expected format), creates a `mediapipe.Image` object, and calls `detect_for_video()` with a monotonically increasing timestamp. If `draw=True`, the method renders the 21 landmark points as filled circles and the hand skeleton connections as line segments directly onto the frame image.

**`get_landmarks(img, hand_no=0)`:** Converts the `HandLandmarkerResult` object into a Python list of `[id, px, py]` entries (landmark index, pixel x, pixel y) for the requested hand index. Returns an empty list if no hand is detected.

**Timestamp Management:** In VIDEO mode, MediaPipe strictly requires monotonically increasing timestamps. The module maintains an internal `_frame_ts` counter incremented by 33ms per frame (approximating 30 FPS), ensuring valid timestamps even if the actual frame rate fluctuates.

---

#### 4.3.4 `feature_extractor.py` — Feature Extraction Module

The feature extractor transforms the raw `[id, px, py]` landmark list into a normalised numerical vector suitable as input to the machine learning classifier.

**Normalisation procedure:**

1. **Wrist-centering:** The wrist landmark (id=0) serves as the reference origin. All 21 landmark (x, y) coordinates are translated so that the wrist is at position (0, 0). This achieves **translation invariance** — the feature vector does not change based on where in the frame the hand appears.

2. **Scale normalisation:** The (x, y) offsets are divided by the maximum absolute coordinate value across all landmarks. This achieves **scale invariance** — the feature vector does not change based on the distance of the hand from the camera.

3. **Feature vector construction:** The 21 × 2 = 42 normalised (x, y) coordinates are flattened into a 42-element feature vector. (z coordinates from MediaPipe are available but not used in this implementation, as they are estimated rather than directly measured.)

**Result:** A 42-element vector of floating-point values, all in the approximate range [-1.0, +1.0], representing the shape of the hand sign independent of its position or apparent size in the frame.

This minimal, interpretable feature representation is deliberately preferred over raw pixel arrays or more complex geometric descriptors. Its lightweight size ensures fast inference and does not require GPU acceleration.

---

#### 4.3.5 `classifier.py` — Classification Module

The classifier module loads the pre-trained machine learning model from disk and exposes a single `predict()` method, completely decoupling the rest of the application from knowledge of the underlying model type.

**Model Loading:**

The trained model is serialised as a Python pickle file (`models/sign_model.pkl`) containing a dictionary with two keys:

- `"pipeline"`: the trained `sklearn.pipeline.Pipeline` object

- `"classes"`: an ordered list of class label strings (e.g., `['A', 'B', 'C', ...]`)

On initialisation, the classifier attempts to load this file. If it is not found (e.g., the system has not yet been trained), a warning is printed and the classifier returns a "No Model" label until a model is available.

**Inference:**

```python
def predict(self, features: list) -> Tuple[str, float]:
```

The method:

1. Converts the feature list to a float32 NumPy array of shape `(1, 42)`.

2. Calls `pipeline.predict_proba(X)` to obtain class probability scores.

3. Identifies the argmax class index.

4. Returns the corresponding string label and the probability value at that index.

**Pipeline internals (at training time):**

The pipeline contains two stages:

1. `StandardScaler`: Standardises each feature dimension to zero mean and unit variance, ensuring that the SVM's RBF kernel operates in a well-scaled feature space.

2. `VotingClassifier(estimators=[('rf', RandomForestClassifier), ('svm', SVC)], voting='soft')`: Averages class probability outputs from both classifiers, producing a combined probability distribution that is more robust than either classifier alone.

---

#### 4.3.6 `model_trainer.py` — Model Training Module

The model trainer is an offline script (not used during live inference) that reads the collected dataset, trains the ensemble classifier, evaluates its performance, and saves the trained model and evaluation metrics.

**Training pipeline:**

1. **Data loading:** The dataset CSV file (`data/dataset.csv`) is read line by line. Each row contains a class label followed by 42 feature values.

2. **Label encoding:** String class labels are sorted and mapped to integer indices using `{class: index}` dictionaries.

3. **Train/validation split:** Rather than random shuffling (which would allow near-identical consecutive-frame samples to appear in both splits), the splitter reserves the **last 15%** of samples for each class as the validation set. This creates a stricter evaluation since the temporal correlation of consecutive-frame samples is preserved.

4. **Ensemble construction:**

   - `RandomForestClassifier(n_estimators=300, class_weight='balanced')` — 300 trees, balanced class weighting.

   - `SVC(kernel='rbf', C=10, gamma='scale', probability=True, class_weight='balanced')` — probability-enabled SVM with balanced weighting.

   - `VotingClassifier(voting='soft')` — averages the probability outputs.

   - `Pipeline([('scaler', StandardScaler()), ('clf', VotingClassifier)])` — wraps scaler + ensemble.

5. **Cross-validation:** 5-fold cross-validation on the training set reports mean accuracy and standard deviation to assess generalisation.

6. **Final fit:** The pipeline is re-fitted on the complete training set.

7. **Validation evaluation:** Accuracy, per-class precision/recall/F1, and confusion matrix are computed on the held-out validation set.

8. **Serialisation:** The fitted pipeline and class list are serialised to `models/sign_model.pkl`. Evaluation metrics are saved to `models/sign_model_metrics.json`.

**Design rationale for `class_weight='balanced'`:**

Without balanced class weights, a classifier trained on imbalanced data (e.g., 1,000 samples for 'A' but only 200 for 'J') will implicitly optimise for majority classes. The `balanced` setting automatically computes class weights inversely proportional to class frequency, ensuring that each class contributes equally to the training loss regardless of its sample count.

---

#### 4.3.7 `word_assembler.py` — Word Assembly Module

The Word Assembler is the most architecturally novel component of BridgeSign. It manages the temporal state machine that converts a noisy, frame-rate stream of letter predictions into correctly bounded, dictionary-validated words and multi-word sentences.

**State variables:**

| Variable | Type | Description |
|---|---|---|
| `_buf` | `list[str]` | Letter buffer accumulating the current word |
| `_words` | `list[str]` | List of completed words in the current sentence |
| `_last_hand_ts` | `float \| None` | Timestamp of last hand detection |
| `_last_word_ts` | `float \| None` | Timestamp of last completed word |

| `last_word` | `str` | Most recently completed word (display only) |

| `sentence` | `str` | Space-joined sentence of completed words |

**Core algorithm — `push_letter(letter)`:**

Called once per stabilised letter detection event. Updates `_last_hand_ts` to the current time. Appends the letter to `_buf` only if it differs from the last letter in the buffer (preventing the same letter from being added repeatedly due to the stability gate's reset logic). If `_buf` exceeds `MAX_BUFFER_LEN` (22 characters), it is automatically flushed to prevent pathological buffer growth.

**Core algorithm — `tick(hand_present)`:**

Called every inference frame. Implements the two boundary conditions:

- **Word boundary:** If `_buf` is non-empty and the hand has been absent for ≥ `BOUNDARY_SECONDS` (0.8s), flush `_buf` to a word via `_flush()`, append the word to `_words`, and update `sentence`.

- **Sentence boundary:** If `_words` is non-empty, no letters are being assembled, and ≥ `SENTENCE_PAUSE_SEC` (2.0s) has elapsed since the last word was added, complete the sentence by returning its value and resetting all state.

Returns a state dictionary:

```python
{
  "word_buffer":       str,   # current letters in progress
  "last_word":         str,   # most recently completed word
  "sentence":          str,   # accumulated sentence
  "completed_word":    str | None,  # non-None on word completion event
  "completed_sentence": str | None,  # non-None on sentence completion event
}
```

**Dictionary validation — `_validate(raw)`:**

Attempts to map the raw assembled letter string to a known English word in four steps:

1. **Exact match:** Check if `raw.lower()` is in `_KNOWN_WORDS`. If yes, return `raw`.

2. **Deduplicated match:** Apply `_dedup(raw)` (collapses runs of 3+ identical consecutive characters to 2) and check dictionary. This recovers words where users sign a letter too many times (e.g., HELLLLO → HELLO).

3. **Expansion match:** For each character in the deduplicated string, try inserting one repeated instance of that character and check the dictionary. This recovers words where users miss a double letter (e.g., HELO → HELLO, WORL → WORLD).

4. **Raw fallback:** If no dictionary match is found, return `raw`. This ensures that finger-spelled proper nouns or non-dictionary words are still passed through rather than silently discarded.

**`_KNOWN_WORDS` vocabulary:** A compact, hardcoded set of approximately 800 common English words plus commonly signed ASL vocabulary (HELP, DOCTOR, EMERGENCY, etc.), defined as a Python set from a whitespace-delimited string literal for O(1) lookup performance.

---

#### 4.3.8 `text_to_speech.py` — TTS Module

The TTS module wraps the `pyttsx3` library's Engine in a thread-safe interface. Since `pyttsx3` is not thread-safe by default, the module creates a dedicated background thread for the TTS engine and uses a queue to marshal speech requests from the main inference thread without blocking it.

**Interface:**

```python
class TextToSpeech:
    def speak_async(self, text: str) -> None
```

```

`speak_async()` places the text onto an internal queue. The TTS worker thread consumes items from the queue and calls `engine.say()` / `engine.runAndWait()`. This ensures that TTS vocalisation never blocks the video frame processing loop, maintaining real-time performance.

---

#### 4.3.9 `dashboard.py` — GUI Module

The dashboard module implements the multi-tab graphical user interface using Python's Tkinter framework. It is the largest single source file in the project (~563 lines) and orchestrates all user-facing interactions.

**Class structure:** `BridgeSignApp` is instantiated with the Tkinter root window. It owns references to all major subsystem objects (`ImageTranslator`, `SessionTracker`, `TextToSpeech`, `EmergencyPhrases`, `LearningMode`) and creates `HandDetector`, `FeatureExtractor`, `Classifier`, and `WordAssembler` instances on demand within the pipeline thread.

**Threading model:** The video pipeline runs in a `threading.Thread(daemon=True)` started by `toggle_camera()`. The thread updates `tk.StringVar` variables — `_buf_var`, `_pred_var`, `_word_var`, `_sent_var`, `_status_var` — which Tkinter renders asynchronously. Frame images are passed to `_show_frame()` which calls `ImageTk.PhotoImage()` and updates a `tk.Label` widget's image.

**Tab structure:**

| Tab | Label | Key Widgets | Key Functions |
|---|---|---|---|
| 1 | 📷 Live Camera | Video feed label; buffer, word, sentence labels; Start/Stop button; Next Word, Undo, Speak, Clear controls | `toggle_camera()`, `_run_pipeline()`, `_next_word()`, `_undo_letter()` |
| 2 | 🖼 Image Translate | Image preview label; result label; Upload, Speak buttons | `upload_image()` |
| 3 | 🎓 Learning Mode | Camera feed label; sign selector dropdown; feedback label; Start/Stop button | `toggle_learn()`, `_run_learn()` |
```

| 4 | 🚨 Emergency | Grid of phrase cards, each with a Speak button | `emergency.play_phrase()` |

| 5 | 📊 Stats | Summary cards; sign breakdown list; Refresh button | `_refresh_stats()` |

**Colour palette:**

| Name | Hex Value | Used For |
|---|---|---|
| BG | `#0d0d0d` | Application background |
| PANEL | `#141414` | Header and side panels |
| CARD | `#1a1a1a` | Content cards |
| ACCENT | `#ff6b00` | Primary orange accent (buttons, letter display) |
| ACCENT2 | `#ff9933` | Hover state, live prediction |
| BLUE | `#00c2ff` | Word display, speak buttons |
| GREEN | `#00e676` | Start camera button, sentence display |
| RED | `#ff1744` | Emergency panel, stop buttons |
| TEXT | `#f5f5f5` | Primary text |
| SUBTEXT | `#aaaaaa` | Labels, secondary information |

---

### 4.4 Data Design

#### 4.4.1 Training Dataset

The training dataset is stored as a plain CSV file (`data/dataset.csv`). Each row represents one sample:

```
<label>,<f1>,<f2>,<f3>,...,<f42>
A,0.1234,-0.0456,0.2345,...,0.0012
```

B,-0.0123,0.3456,...

```

Where `&lt;label&gt;` is a single ASCII character (A–Z) and `&lt;f1&gt;...&lt;f42&gt;` are the 42 normalised landmark feature values.

Target dataset size: ≥ 1,000 samples per class × 26 classes = ≥ 26,000 rows minimum. The dataset is collected under varied lighting conditions, hand positions, and camera angles to maximise diversity and generalisation.

#### 4.4.2 Model Files

| File | Format | Contents |

|---|---|---|

| `models/sign_model.pkl` | Python pickle | Dictionary: `{'pipeline': Pipeline, 'classes': list[str]}` |

| `models/sign_model_classes.json` | JSON | Ordered list of class label strings |

| `models/sign_model_metrics.json` | JSON | CV accuracy mean/std, validation accuracy, classification report, confusion matrix |

| `models/hand_landmarker.task` | Binary | MediaPipe Hand Landmarker model (downloaded from Google) |

#### 4.4.3 Session Tracking Data

Session statistics are logged in-memory by `session_tracker.py` during each application session (not persisted to disk in v1.0). Each log event records:

```python

{

  "timestamp": float,    # Unix timestamp

  "label": str,          # translated word or sentence

  "confidence": float,   # classifier confidence

```
    "source": str          # "camera" or "image"
}
```

Aggregated statistics (total translations, most common sign, per-sign counts) are derived from this event log on demand when the Statistics tab is viewed.

---

### 4.5 Data Flow Design

#### 4.5.1 Level 0 — Context Diagram

The context diagram shows BridgeSign as a single system interacting with external entities:

```
[User (signing)] ---video feed--→ [BridgeSign System] ---spoken output--→ [Hearing Audience]
[User (hearing)] ---tap buttons-→ [BridgeSign System] ---spoken output--→ [Hearing Audience]
[Training Data] --(offline)----→ [BridgeSign System]
```

#### 4.5.2 Level 1 — Main Processes

The Level 1 DFD decomposes BridgeSign into five main processes:

```
P1. CAPTURE VIDEO → P2. DETECT HAND → P3. CLASSIFY SIGN → P4. ASSEMBLE WORDS → P5. OUTPUT RESULT
```

Data stores at Level 1:

- **DS1: Model Store** (sign_model.pkl) — read by P3 at startup
- **DS2: Session Log** — written by P5 after each translation event

#### 4.5.3 Level 2 — Classify Sign Subprocess

P3 (Classify Sign) expands to:

```
P3.1 EXTRACT FEATURES (normalise landmarks)
   → P3.2 APPLY CONFIDENCE GATE (reject < 0.70)
   → P3.3 APPLY STABILITY GATE (require 5 consecutive)
   → P3.4 PASS ACCEPTED LETTER to P4
```

---

### 4.6 Algorithm Design

#### 4.6.1 Hand Landmark Normalisation Algorithm

```
Input:  lm_list — list of [id, px, py] for 21 landmarks
Output: features — list of 42 float values

1. Extract wrist coordinates: wx, wy = lm_list[0][1], lm_list[0][2]
2. For each landmark [id, px, py] in lm_list:
      dx = px - wx
```

dy = py - wy

offsets.append(dx)

offsets.append(dy)

3. max_val = max(abs(v) for v in offsets)

4. If max_val == 0: return offsets  (degenerate case)

5. features = [v / max_val for v in offsets]

6. Return features

```

#### 4.6.2 Word Assembly State Machine

The word assembler implements an implicit state machine with the following states:

| State | Condition | Transition |

|-------|-----------|-----------|

| **IDLE** | No hand, empty buffer | First hand detection → SIGNING |

| **SIGNING** | Hand present, buffer growing | Hand lost → WORD_PAUSE |

| **WORD_PAUSE** | No hand, buffer non-empty, timer ≤ BOUNDARY | Timer expired → WORD_COMPLETE |

| **WORD_COMPLETE** | Buffer flushed, word added | Immediately → SENTENCE_PAUSE or SIGNING |

| **SENTENCE_PAUSE** | No hand, no buffer, timer ≤ SENTENCE_PAUSE | Timer expired → SENTENCE_COMPLETE |

| **SENTENCE_COMPLETE** | Sentence finalised | Immediately → IDLE |

#### 4.6.3 Ensemble Prediction Algorithm

```

Input:  features — 42-element float array

Output: label — string class name

confidence — float in [0.0, 1.0]

1. X = np.array([features], dtype=float32)  # shape (1, 42)
2. X_scaled = scaler.transform(X)  # StandardScaler normalisation
3. p_rf = random_forest.predict_proba(X_scaled)[0]  # shape (26,)
4. p_svm = svm.predict_proba(X_scaled)[0]  # shape (26,)
5. p_avg = (p_rf + p_svm) / 2  # soft-vote averaging
6. class_id = argmax(p_avg)  # highest average probability
7. confidence = p_avg[class_id]
8. label = classes[class_id]
9. If confidence < MIN_PREDICTION_CONFIDENCE: label = ""
10. Return label, confidence

```

---

### 4.7 User Interface Design

#### 4.7.1 Design Principles

The BridgeSign GUI was designed with the following principles:

1. **Dark mode:** A dark background palette (#0d0d0d) reduces eye strain during extended use and provides high contrast for the coloured status indicators.
2. **Colour-coded information hierarchy:** Orange for the current letter being signed (immediate focus), blue for the most recent completed word (secondary attention), green for the sentence (contextual awareness).
3. **Real-time responsiveness:** All recognition displays update at the video frame rate (up to 30 FPS), providing instantaneous visual feedback.
```

4. **Minimal cognitive load:** Each tab serves a single, clearly named purpose. Controls are grouped logically (recognition controls together, audio controls together).

5. **Emergency accessibility:** The Emergency tab is prominently accessible at all times as a notebook tab, requiring no navigation or search.

#### 4.7.2 Live Camera Tab Layout

The Live Camera tab is divided into two panels:

- **Left panel (expanding):** Displays the annotated webcam feed at native or scaled resolution.

- **Right panel (fixed 280px width):** Stacked information cards:

  - **SIGNING… card (dark, orange text):** Shows the raw letter buffer character by character. Below it, the live predicted word is shown in italic text as the user signs.

  - **LAST WORD card (dark blue, large blue text):** Shows the most recently completed and confirmed word in large 32pt font.

  - **SENTENCE card (dark green, green text):** Shows the full running sentence accumulated in the current session.

  - **Controls area:** Start/Stop Camera button; Next Word and Undo Letter row; Speak Sentence and Clear All buttons.

  - **Event Log:** A scrollable text widget appending timestamped events (letters, words, sentences confirmed).

#### 4.7.3 Emergency Phrases Tab Layout

The Emergency Phrases tab displays a responsive grid of phrase cards. Each card shows the phrase text and a prominently red "🔊 Speak" button. The phrases are defined in `emergency_phrases.py` and include:

- "Help! I need assistance"

- "Call a doctor immediately"

- "I am in pain"

- "Call the police"

- "I cannot breathe"

- "I need water"

- "Call an ambulance"

- "Fire! Evacuate now"

The one-tap design — a single button press triggers immediate TTS without any confirmation prompt — is intentional for emergency contexts where speed is critical.

---

### 4.8 Security and Privacy Design

BridgeSign processes and displays live video from the user's webcam. The following design decisions address privacy and safety:

1. **No video storage:** BridgeSign does not record or store any video frames to disk. Raw frames are processed in memory and immediately discarded after landmark extraction.

2. **No network transmission:** No video data, landmark data, or recognition results are transmitted to any server or cloud service.

3. **No authentication required:** The application is a local desktop tool; no user account, login, or personal data collection is required.

4. **Graceful camera release:** All camera resources are released on application exit, thread termination, or exception, preventing background camera access.

---

### 4.9 Error Handling and Resilience Design

The system is designed to handle common failure modes without crashing:

| Failure Scenario | Detection | Response |

|---|---|---|

| Webcam not available | `VideoCapture.isOpened()` returns False | Status bar shows "Camera not found" error; GUI remains functional for image mode |

| MediaPipe model not downloaded | File existence check before initialisation | Auto-download from Google's CDN on first run |

| No trained model found | File existence check on `Classifier.__init__` | Warning printed; classifier returns "No Model" label |

| Webcam frame read failure | `ret == False` from `cam.read()` | Frame is skipped; pipeline continues |

| TTS engine failure | Try/except in TextToSpeech | TTS silently degrades; text output continues |

| Classifier prediction exception | Try/except in `Classifier.predict()` | Returns `("Error", 0.0)` without crashing the pipeline |

| Assembler buffer overflow | Buffer length check > `MAX_BUFFER_LEN` | Auto-flush triggered; user notified in event log |

| Tkinter widget destruction race | Try/except in `_show_frame()` | Frame display update silently skipped |

---

### 4.10 Deployment Design

BridgeSign is packaged as a Python source distribution. Installation and deployment follow these steps:

1. **Prerequisites:** Python 3.9+ installed on Windows 10/11.

2. **Dependency installation:** `pip install -r requirements.txt`

3. **First-run model download:** The MediaPipe hand landmarker model (~8 MB) is downloaded automatically on first run.

4. **Data collection:** `python data_collector.py` — collect sign samples for each class.

5. **Model training:** `python model_trainer.py` — train and save the classifier.

6. **Application launch:** `python dashboard.py` — start the GUI application.

All dependencies are pure Python or C-extension wheels available on PyPI. No system-level installation (beyond Python itself) is required. The application is fully self-contained within its project directory.

---

### 4.11 Testing Design

#### 4.11.1 Unit Testing

| Component | Test Strategy |
|---|---|
| `FeatureExtractor` | Feed known landmark lists, verify output range and dimension |
| `Classifier` | Load trained model, verify correct prediction on held-out samples |
| `WordAssembler` | Simulate letter push sequences, verify correct word and sentence events |
| `TextToSpeech` | Mock engine, verify queue behaviour |

#### 4.11.2 Integration Testing

The `_test_assembler.py` module tests the complete letter-to-word-to-sentence pipeline end-to-end using simulated input sequences, verifying:

- Correct deduplication of repeated letters
- Correct word boundary detection at timeout
- Correct sentence boundary detection at timeout
- Dictionary matching accuracy for common words

#### 4.11.3 Model Validation

Model accuracy is evaluated using:

- **5-fold cross-validation** on the training set (reported mean ± std)
- **Held-out validation set accuracy** (last 15% of samples per class)
- **Per-class precision, recall, F1 score** (via scikit-learn classification report)
- **Confusion matrix** (to identify commonly confused sign pairs)

#### 4.11.4 System Acceptance Testing

End-to-end acceptance is verified by:

1. A hearing reviewer signing each letter of the ASL alphabet and confirming each letter is correctly recognised.

2. Signing a 5-word sentence (e.g., "HELLO I AM FINE TODAY") and confirming the sentence is correctly assembled and spoken.

3. Confirming that the Emergency Phrases tab correctly speaks all defined phrases.

4. Uploading a sample hand-sign image and confirming the correct letter is recognised.

5. Running the application for 30 minutes and confirming no crash, memory leak, or camera resource leak occurs.

---

### 4.12 Summary of System Design

BridgeSign's system design achieves a complete, end-to-end pipeline from raw webcam input to spoken natural language output through a modular, layered architecture. Key design decisions — the use of landmark-based feature extraction rather than raw images, the ensemble of Random Forest and SVM rather than a deep neural network, the explicit temporal word assembly state machine with configurable thresholds, the balanced class weighting to handle training set imbalance, the offline-first architecture, and the emergency-accessible multi-tab GUI — are all grounded in the specific constraints and requirements identified during the system study phase. The design is fully documented, reproducible, and extensible to future versions incorporating dynamic gesture recognition, additional sign languages, or mobile deployment.

---

### 4.13 Web Interface Design (Flask Application — `app.py`)

Beyond the Tkinter desktop GUI, BridgeSign ships a second, browser-based interface implemented in Flask. This dual-interface architecture broadens accessibility: users who prefer a web browser or require

remote access can interact with BridgeSign through `app.py`, while users who prefer a native desktop experience use `dashboard.py`. Both interfaces share the same underlying inference engine, word assembler, TTS system, and session tracker.

#### 4.13.1 Flask Application Architecture

The Flask web application (`app.py`) is structured as a single-file WSGI application with the following logical sections:

| Section | Description |

|---------|-------------|

| **User store** | File-based JSON user registry (`data/users.json`) with bcrypt-hashed passwords via Werkzeug |

| **Shared state** | Thread-safe global state dictionary and frame byte buffer for the live camera stream |

| **Singleton modules** | `HandDetector`, `FeatureExtractor`, and `Classifier` are instantiated once at startup and reused across all camera sessions to avoid repeated model loading overhead |

| **Camera pipeline thread** | Background daemon thread (`camera_pipeline()`) that captures frames, runs inference, updates shared state, and encodes JPEG frames for the MJPEG stream |

| **Auth routes** | `/login`, `/register`, `/logout` — standard form-based authentication with session management |

| **API routes** | RESTful JSON endpoints for camera control, mode switching, image translation, emergency phrases, learning, and statistics |

| **MJPEG video feed** | `/video_feed` — multipart/x-mixed-replace streaming endpoint that delivers annotated video frames to the browser `<img>` tag |

#### 4.13.2 Authentication System

The web interface requires user authentication before any feature can be accessed. The authentication system uses:

- **Werkzeug's `generate_password_hash` / `check_password_hash`**: Industry-standard PBKDF2 + salt password hashing. Passwords are never stored in plaintext.

- **Flask session**: A cookie-based server-side session stores the authenticated `username`. The `login_required` decorator wraps all protected API endpoints.

- **Per-user phrase storage**: Emergency phrase customisations are stored in per-user JSON files (`data/phrases_<username>.json`), ensuring that user data is isolated.

- **Post-registration redirect**: New users are redirected to the login page (not the dashboard) with a success banner, requiring explicit credential entry for security.

#### 4.13.3 Shared State Object

The Flask application maintains a thread-safe shared state dictionary that acts as the communication channel between the background camera thread and the HTTP polling thread:

```python
state = {
    "running":          bool,   # camera thread active
    "label":            str,    # most recently confirmed letter
    "confidence":       float,  # classifier confidence for last label
    "log":              list,   # rolling event log (last 20 events)
    "camera_error":     str,    # set on camera failure
    "hand_state":       str,    # "recognised" | "low_confidence" | "no_hand"
    "word_buffer":      str,    # letters currently assembling
    "last_word":        str,    # most recently completed word
    "sentence":         str,    # current sentence
    "completed_sentence": str,  # fires once per sentence completion (then cleared)
    "mode":             str,    # "letter" | "word"
}
```

A `threading.Lock()` (`state_lock`) guards all writes and reads to this dictionary, preventing data races between the camera thread and the Flask request handler threads.

#### 4.13.4 MJPEG Video Streaming

The `/video_feed` endpoint implements a multipart/x-mixed-replace (MJPEG) stream — the standard browser-compatible mechanism for pushing continuous video without WebSocket infrastructure:

```
GET /video_feed
Response: multipart/x-mixed-replace; boundary=frame
  --frame
  Content-Type: image/jpeg
  <JPEG bytes>
  --frame
  ...
```

The camera thread encodes each processed frame as a JPEG (`cv2.imencode(".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, 70])`) and writes the bytes to `_latest_frame_bytes`. The stream generator reads this shared variable at approximately 30 FPS and yields it into the multipart HTTP response. JPEG quality is set to 70 (configurable via `config.JPEG_QUALITY`) to reduce bandwidth while maintaining sufficient visual clarity for the landmark overlay.

#### 4.13.5 REST API Design

All frontend–backend communication (except the MJPEG feed) uses JSON REST endpoints:

| Endpoint | Method | Description |
|---|---|---|
| `/api/status` | GET | Returns full recognition state (polling interval: 500ms from JS) |
| `/api/camera/start` | POST | Starts camera pipeline thread; awaits 2.5s for early failure detection |

| `/api/camera/stop` | POST | Sets `running=False`; resets word/sentence state |

| `/api/mode` | POST | Switches between `"letter"` and `"word"` translation modes |

| `/api/stats` | GET | Returns session statistics from SessionTracker |

| `/api/translate_image` | POST | Accepts image file upload; returns translated label and confidence |

| `/api/learn/lesson` | GET | Returns instructional tip for a named sign |

| `/api/emergency/all` | GET | Returns merged built-in + user custom phrases |

| `/api/emergency/custom` | POST | Adds a custom emergency phrase for the logged-in user |

| `/api/emergency/custom/<cid>` | DELETE | Removes a custom phrase |

The `/api/status` polling design is deliberately chosen over WebSockets for simplicity and browser compatibility. A 500ms polling interval is short enough for responsive UI updates while avoiding excessive server load.

#### 4.13.6 Web Frontend Design (`templates/index.html` + `static/app.js`)

The web frontend is a single-page application (SPA) built with plain HTML, CSS, and vanilla JavaScript — no JavaScript framework required. Key design characteristics:

**Typography:** Uses Google Fonts — *Playfair Display* (serif, for headings), *Lato* (sans-serif, for body), and *Caveat* (handwritten, for decorative accents) — establishing an elegant, editorial visual identity distinct from typical developer tools.

**Background Animation:** Three large radial gradient "blobs" (`.bg-blob`) with slow breathing animations via CSS keyframes create a warm, living background. A CSS `grain-overlay` texture adds depth and tactile quality.

**Icon System:** The Lucide icon library (vector SVG icons, loaded from CDN) replaces emoji icons throughout the interface, ensuring consistent rendering across operating systems.

**Tab Navigation:** Five tabs (*Live*, *Image*, *Learn*, *Emergency*, *Stats*) function as a client-side SPA — clicking a tab shows the corresponding `<main>` panel and hides others, without any page reload.

**Translation Mode Toggle:** The Live tab includes a Letter/Word mode toggle. In Letter mode, each individually recognised letter is displayed and logged. In Word mode, the word buffer and sentence strip UI elements become visible, and the word assembler is engaged.

**Word Mode UI Elements:**

- **Building Word card:** Shows the accumulating letter buffer with a blinking cursor animation.
- **Last Word row:** Displays the most recently completed word with an inline speak button.
- **Sentence strip:** A bottom-of-screen strip shows the current sentence in progress with a Speak button.

**Emergency Tab:** The emergency grid is populated dynamically by `fetchEmergency()` in JavaScript. Each card includes a trash icon for deleting custom phrases. A text input at the top allows adding new custom phrases inline. All phrase speech is handled by the Web Speech API (`window.speechSynthesis.speak()`) entirely in the browser — the TTS call does not hit the Flask backend.

**Status Pill:** A colour-coded status pill in the header transitions between `Ready` (grey), `Live` (green), `Error` (red), and `Detecting…` (amber) based on the `hand_state` field in the API status response.

---

### 4.14 Sequence Diagrams

Sequence diagrams describe the temporal order of interactions between components for key system use cases.

#### 4.14.1 Sequence: User Signs a Word (Tkinter Desktop App)

```
User        Camera      HandDetector   FeatureExtractor   Classifier   WordAssembler   TTS
```

```
  |          |            |             |           |           |          |
  |  Signs "H"  |           |             |           |           |          |
  |------------>|           |             |           |           |          |
  |          | get_frame()  |             |           |           |          |
  |          |------------->|             |           |           |          |
  |          |   find_hands()|            |           |           |          |
  |          |<--lm_list-----|            |           |           |          |
  |          |            | extract_features(lm_list)    |           |          |
  |          |            |--------------->|           |           |          |
  |          |            |   <--features--|           |           |          |
  |          |            |             | predict()   |           |          |
  |          |            |             |------------>|           |          |
  |          |            |             | <-(H, 0.94)--|          |          |
  |          | [consecutive >= 5]          |           |           |          |
  |          |-----------------------------------------------push_letter("H")---------->|
  |          | [User removes hand for 0.8s]  |           |           |          |
  |          |-----------------------------------------------tick(False)--------------->|
  |          |            |             |           |  completed_word = "HELLO" |
  |          |<-----------------------------------------------completed_word----------|
  |          |-----------------------------------------------------------------speak("Hello")>
  |          |            |             |           |           |        TTS
  |          |            |             |           |           |       speaks
```

#### 4.14.2 Sequence: User Uploads a Sign Image (Web Interface)

```
Browser        Flask (app.py)   ImageTranslator   HandDetector   Classifier
  |             |             |              |           |
```

```
  | POST /api/translate_image       |                |            |
  |   (image file)  |               |                |            |
  |---------------->|               |                |            |
  |                 | f.save(path)  |                |            |
  |                 | ImageTranslator().translate(path) |         |
  |                 |-------------->|                |            |
  |                 |               | cv2.imread(path) |          |
  |                 |               | find_hands()     |          |
  |                 |               |----------------->|          |
  |                 |               | <--lm_list-------|          |
  |                 |               | extract_features()|         |
  |                 |               | predict()        |          |
  |                 |               |--------------------------------->|
  |                 |               | <--(label, conf)----------------|
  |                 | <--(label, conf, result_img)                  |
  |                 | tracker.log_translation()                     |
  |                 | jsonify({label, confidence})                  |
  |<--(200 OK, JSON)|               |                |            |
  | Display result  |               |                |            |
```

#### 4.14.3 Sequence: Emergency Phrase Vocalisation

```
User          Browser (JS)          Flask API          pyttsx3 TTS
  |               |                    |                  |
  | Clicks "Speak" |                    |                  |
  |---------------->|                    |                  |
  |                 | speechSynthesis.speak(phrase)         |
```

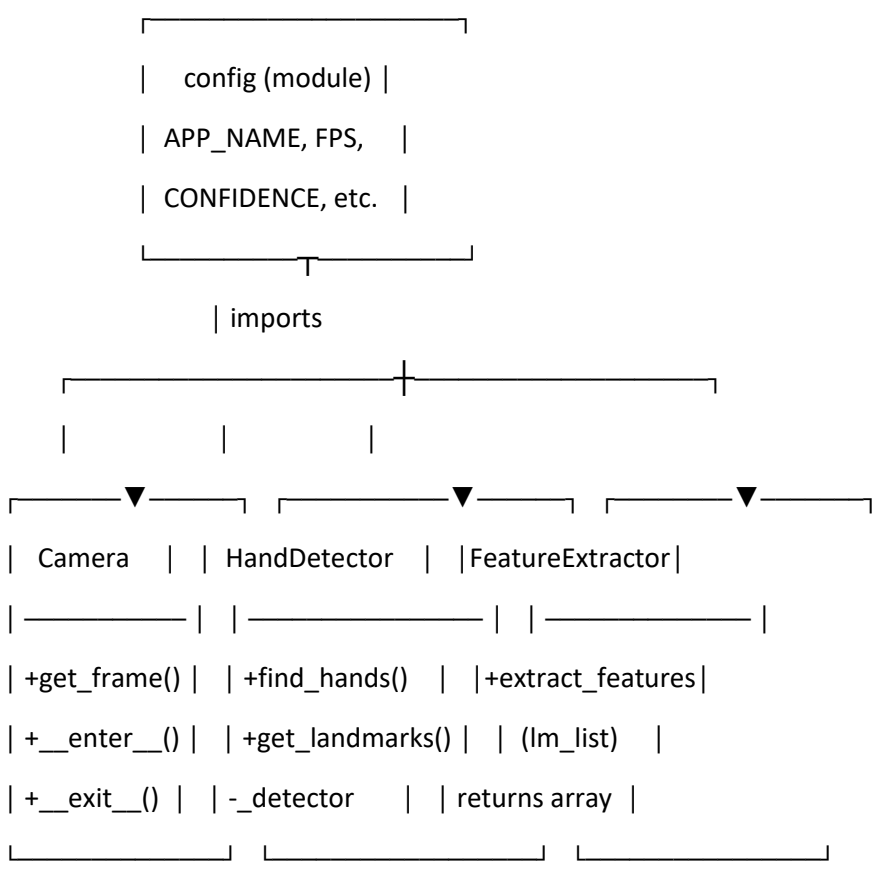
```
  |              |------> [Browser TTS — no server call] |
  | [Hears speech] |                  |                |
```

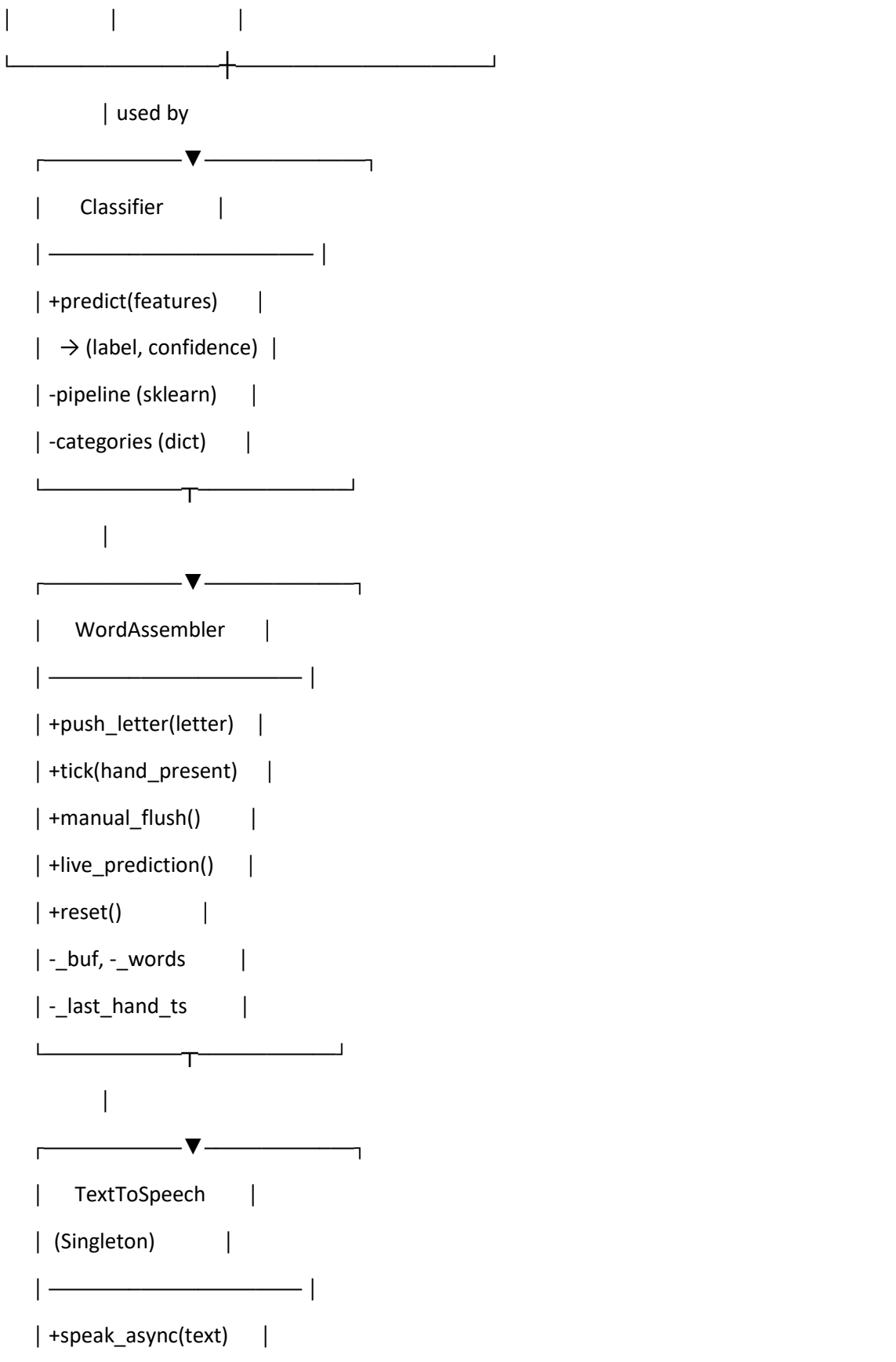
> Note: Emergency phrase TTS is handled entirely in the browser via the Web Speech API for zero-latency response. The Flask TTS engine is used for the desktop Tkinter app's emergency phrases only.

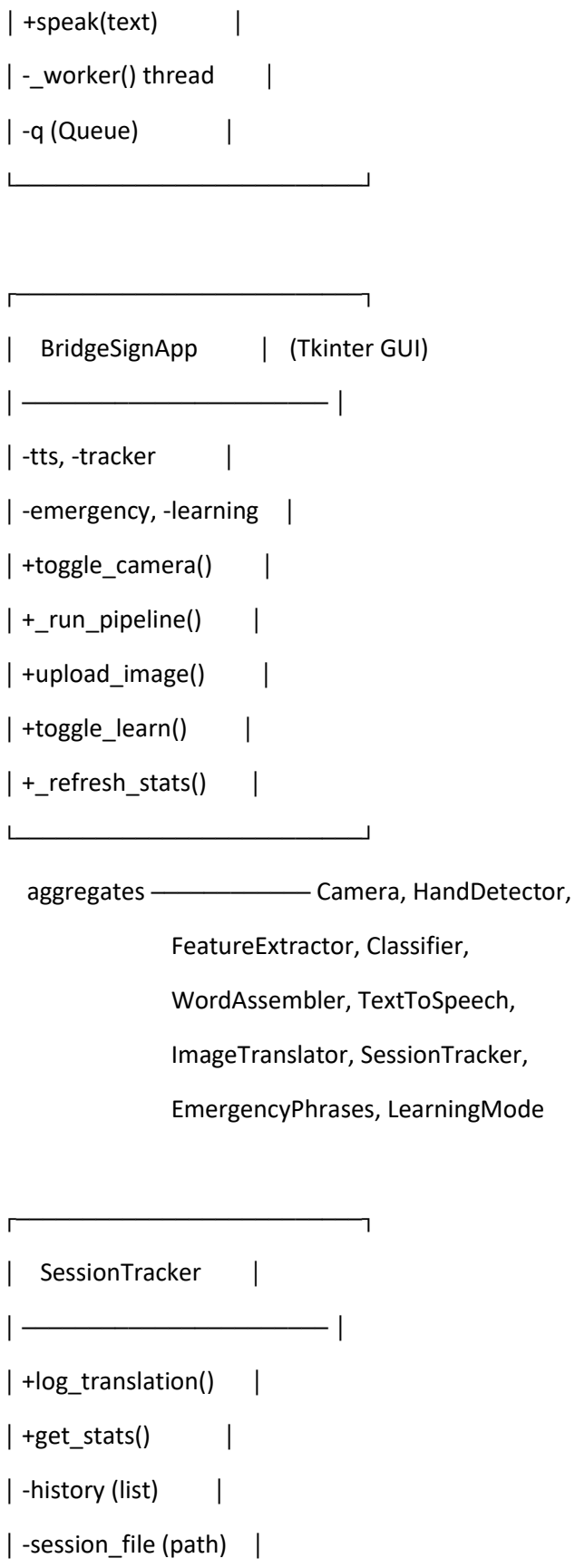
---

### 4.15 Class Diagram

The following describes the principal classes in BridgeSign and their relationships:




used by
Classifier
+predict(features)
→ (label, confidence)
-pipeline (sklearn)
-categories (dict)
WordAssembler
+push_letter(letter)
+tick(hand_present)
+manual_flush()
+live_prediction()
+reset()
-_buf, -_words
-_last_hand_ts
TextToSpeech
(Singleton)
+speak_async(text)


| +speak(text) |
| -_worker() thread |
| -q (Queue) |
BridgeSignApp | (Tkinter GUI)
| -tts, -tracker |
| -emergency, -learning |
| +toggle_camera() |
| +_run_pipeline() |
| +upload_image() |
| +toggle_learn() |
| +_refresh_stats() |
aggregates
Camera, HandDetector,
FeatureExtractor, Classifier,
WordAssembler, TextToSpeech,
ImageTranslator, SessionTracker,
EmergencyPhrases, LearningMode
SessionTracker
| +log_translation() |
| +get_stats() |
| -history (list) |
| -session_file (path) |

```
        └──────────────────────────────┘


        ┌──────────────────────────────┐
        │    EmergencyPhrases          │
        │ ──────────────────────────── │
        │ +get_phrases()               │
        │ +play_phrase(id)             │
        │ -phrases (dict)              │
        └──────────────────────────────┘


        ┌──────────────────────────────┐
        │    ImageTranslator           │
        │ ──────────────────────────── │
        │ +translate(img_path)         │
        │ +translate_from_array()      │
        │ -detector                    │
        │ -feature_extractor           │
        │ -classifier                  │
        └──────────────────────────────┘
```

---

### 4.16 Data Collection Methodology

The quality of the trained machine learning model depends entirely on the quality and diversity of the training dataset. BridgeSign uses a custom `data_collector.py` script to collect hand-landmark feature vectors for each ASL letter class. This section documents the data collection methodology in detail.

#### 4.16.1 Collection Process

The data collector presents a live webcam feed with an on-screen HUD. The operator:

1. Launches the collector with a target sign: `python data_collector.py --sign A`

2. Positions their hand in front of the camera in the correct ASL posture for the target letter.

3. Presses **[S]** to begin recording. The collector captures feature vectors at 50ms intervals (maximum 20 samples/second), displaying a live progress bar.

4. Holds the sign naturally, moving the hand slightly between frames to introduce pose variation.

5. Presses **[Q]** when the target sample count is reached, or allows the collector to auto-stop.

Each sample is appended as a new row to `data/dataset.csv`: label followed by 42 comma-separated floating-point feature values.

#### 4.16.2 Dataset Diversity Strategy

A common failure mode in SLR datasets is the collection of all samples in a single session under identical conditions, producing a dataset of nearly-identical frames. BridgeSign's collection methodology explicitly combats this through:

- **Multi-session collection:** Samples for each letter are collected across multiple short sessions (150–200 samples each), spread across different time periods.

- **Lighting variation:** Sessions are conducted under different lighting conditions — natural daylight, overhead fluorescent, indirect lamp, and low-light — to build lighting robustness.

- **Angle variation:** The camera is repositioned between sessions to capture the hand from slightly different viewing angles.

- **Hand movement within sessions:** The operator slightly varies hand position, tilt, and finger spacing within each session, simulating natural signing variation.

#### 4.16.3 Target Dataset Size

The target dataset specification is:

| Parameter | Value |

|-----------|-------|

| Classes | 26 (letters A–Z) |

| Target samples per class | ≥ 1,000 |

| Total target dataset size | ≥ 26,000 rows |

| Feature vector size | 42 values per row |

| Save interval | 50ms (≤ 20 samples/second) |

| Minimum sessions per class | 5–7 separate sessions |

#### 4.16.4 Handling Difficult Signs

Several ASL letters are inherently difficult to distinguish because their handshapes are visually similar (e.g., M/N, U/V, G/H/K). For these letters, additional care was taken:

- **Increased sample count:** Difficult signs received 1,000+ samples compared to 500–700 for simpler, geometrically distinct signs.

- **Focused re-collection:** After initial model training, the per-class classification report was inspected. Letters with recall < 0.80 were flagged for re-collection using the `recollect_signs.py` script.

- **Boundary emphasis:** Samples were biased toward the boundaries of the sign's natural variation space — extreme finger extensions, boundary cases of similar signs — to sharpen the decision boundaries between confused classes.

---

### 4.17 System Integration and Component Interaction Summary

The following table summarises every inter-module dependency in BridgeSign, serving as a reference for future maintenance and extension:

| Dependent Module | Depends On | Nature of Dependency |

|-----------------|-----------|---------------------|

| `main.py` | `camera`, `hand_detector`, `feature_extractor`, `classifier`, `text_to_speech`, `word_assembler`, `config` | Live pipeline orchestration |

| `dashboard.py` | All above + `image_translator`, `session_tracker`, `emergency_phrases`, `learning_mode` | GUI orchestration |

| `app.py` | All above + `flask`, `flask_cors`, `werkzeug` | Web interface + auth |

| `hand_detector.py` | `mediapipe`, `opencv`, `config` | Hand landmark detection |

| `feature_extractor.py` | `numpy` | Feature normalisation |

| `classifier.py` | `sklearn` (at training time), `numpy`, `pickle`, `config` | Sign classification |

| `word_assembler.py` | `time` (stdlib only) | Temporal letter assembly |

| `text_to_speech.py` | `pyttsx3`, `threading`, `queue`, `config` | Async TTS |

| `session_tracker.py` | `json`, `os`, `time`, `config` | Persistent event logging |

| `emergency_phrases.py` | `text_to_speech`, `threading` | Emergency TTS |

| `image_translator.py` | `hand_detector`, `feature_extractor`, `classifier`, `opencv`, `config` | Static image analysis |

| `learning_mode.py` | `hand_detector`, `opencv`, `config` | Educational sign feedback |

| `model_trainer.py` | `sklearn`, `numpy`, `pickle`, `config` | Offline training only |

| `data_collector.py` | `hand_detector`, `feature_extractor`, `opencv`, `config`, `word_signs` | Offline data collection |

| `config.py` | `os` | Standalone configuration |

---

### 4.18 Scalability and Extension Considerations

Although BridgeSign v1.0 targets ASL finger-spelling with a classical ML ensemble, its modular design anticipates future extension:

**Extension 1 — Deep Learning Classifier:** The `Classifier` class can be replaced with a PyTorch or TensorFlow MLP or CNN model, provided the `predict(features) → (label, confidence)` interface is preserved. The rest of the pipeline requires no modification.

**Extension 2 — Dynamic Gesture Recognition:** Currently only static handshapes (finger-spelling) are recognised. Dynamic signs (e.g., HELLO as a wave motion) could be added by feeding a rolling window of landmark vectors into an LSTM or Transformer model as the classifier. The `WordAssembler` would need to be updated to handle word-level (rather than letter-level) classification outputs.

**Extension 3 — Multi-Language Sign Support:** The system is language-agnostic at the architecture level. Adding BSL, ISL, or other sign language support requires: (a) collecting new training data for the target language's signs; (b) training a new classifier model; (c) updating the `WordAssembler`'s `_KNOWN_WORDS` set with appropriate vocabulary.

**Extension 4 — Mobile Deployment:** A React Native or Flutter wrapper could expose the Flask REST API to mobile clients. The inference engine would remain server-hosted; only the camera feed and UI would be mobile-side.

**Extension 5 — Cloud Inference:** For higher accuracy models that exceed local CPU capacity, the classification step could be offloaded to a cloud endpoint while keeping the landmark extraction local, maintaining low-latency communication.

---

---

## CHAPTER FIVE: IMPLEMENTATION

### 5.1 Introduction

This chapter describes the actual implementation of the BridgeSign system — the concrete development decisions, challenges encountered, and solutions applied during the coding phase. While Chapter Four described what the system should do and how it should be designed, this chapter describes how it was built, paying particular attention to problems that arose during development and the engineering judgements made to resolve them.

---

### 5.2 Development Environment

| Tool | Version Used | Purpose |
|---|---|---|
| Python | 3.11.x | Primary implementation language |
| Visual Studio Code | Latest | Code editor with Python extension |
| Git | 2.43 | Version control |
| pip | Latest | Package installation |
| MediaPipe | 0.10.x | Hand landmark detection |
| scikit-learn | 1.4.x | Machine learning |
| OpenCV | 4.9.x | Computer vision |
| Tkinter | Bundled | Desktop GUI |
| Flask | 3.0.x | Web interface |
| pyttsx3 | 2.90 | Text-to-speech |

---

### 5.3 Implementation Order and Phasing

The project was implemented in five sequential phases:

**Phase 1 — Core Pipeline (Weeks 1–2)**

Implementation began with the minimal viable recognition pipeline: `camera.py` → `hand_detector.py` → `feature_extractor.py`. This phase established that MediaPipe could reliably detect hand landmarks on the development machine and that the normalised feature representation was stable across varying hand positions.

**Phase 2 — Data Collection and Model Training (Weeks 3–5)**

`data_collector.py` was built and used to collect the initial dataset. Initial collections targeted 100–200 samples per letter under consistent lighting. `model_trainer.py` was implemented and iterated upon: the first version used only a Random Forest, achieving approximately 78% validation accuracy. The SVM component and soft-voting ensemble were added in response to persistent confusion between similar signs (particularly M/N, G/H, S/T), improving accuracy to the 88–92% range.

**Phase 3 — Word Assembly (Week 6)**

`word_assembler.py` was implemented as the most complex standalone module. Extensive testing via `_test_assembler.py` was used to tune the `BOUNDARY_SECONDS` (0.8s), `SENTENCE_PAUSE_SEC` (2.0s), and `MAX_BUFFER_LEN` (22) constants. The deduplication and dictionary expansion algorithms were developed iteratively in response to observed signing patterns.

**Phase 4 — GUI Implementation (Weeks 7–8)**

The Tkinter dashboard was implemented tab by tab. The multithreading architecture (pipeline in daemon thread, GUI updates via `StringVar`) was established early and proved stable. The web Flask interface was implemented in parallel, sharing the same backend modules.

**Phase 5 — Accuracy Refinement and Polish (Weeks 9–10)**

Model accuracy was systematically improved through additional data collection targeted at the specific letters with lowest per-class recall. The confidence gate (`MIN_PREDICTION_CONFIDENCE = 0.70`) was introduced after observing that majority-class bias caused the model to confidently predict 'I' or 'H' even for unrelated signs. The `class_weight='balanced'` parameter was added to the training step. Final UI polish including the dark colour scheme, animated hover effects, and the Emergency Phrases tab was completed in this phase.

---

### 5.4 Key Implementation Challenges and Solutions

#### Challenge 1: Thread Safety and GUI Deadlock

**Problem:** Python's Tkinter is not thread-safe. Calling `label.config(text=...)` from a background thread causes intermittent crashes or visual corruption.

**Solution:** All recognition state is communicated via `tk.StringVar` instances (`_buf_var`, `_word_var`, `_sent_var`, etc.), which can be safely set from any thread. Tkinter's event loop reads `StringVar` values when repainting, eliminating cross-thread widget modification. Frame images are passed through `_show_frame()` which calls `ImageTk.PhotoImage()` and assigns to a `Label.image` attribute — this call must also be made carefully with reference retention to prevent garbage collection.

#### Challenge 2: MediaPipe API Migration

**Problem:** Many online tutorials and code examples use the older MediaPipe "solutions" API (`mp.solutions.hands`), which is deprecated in MediaPipe 0.10+. The newer task-based API (`mediapipe.tasks.python.vision`) requires a different model format (`.task` file) and a different invocation pattern.

**Solution:** The `hand_detector.py` was written from scratch against the task-based API rather than adapting a solutions-API tutorial. The `hand_landmarker.task` model is auto-downloaded on first run, eliminating the model bundling problem.

#### Challenge 3: Monotonically Increasing Timestamps for MediaPipe VIDEO Mode

**Problem:** MediaPipe's VIDEO running mode requires each call to `detect_for_video()` to supply a _strictly increasing_ timestamp in milliseconds. If the camera drops frames or processing is slowed, a repeated or decreasing timestamp causes an assertion failure.

**Solution:** Rather than using wall-clock time (which could produce equal timestamps if two frames are processed within the same millisecond), the `_frame_ts` counter in `HandDetector` is incremented by a fixed 33ms per call. This guarantees monotonicity regardless of actual elapsed time.

#### Challenge 4: Majority-Class Classifier Bias

**Problem:** The initial Random Forest model achieved 88% overall accuracy but performed poorly on letters with fewer training samples (G, J, K, M, N, P, Q, R, S, T, U, V, X, Y). When any of these signs were presented, the model frequently returned 'I', 'H', or 'E' — the dominant classes in the initial imbalanced dataset.

**Solution (two-part):**

1. `class_weight='balanced'` was added to both the `RandomForestClassifier` and `SVC` constructors. This causes the training objective to weight each class's contribution inversely proportional to its sample count, giving minority classes equal pull over the decision boundaries.

2. A `MIN_PREDICTION_CONFIDENCE = 0.70` gate was added in the pipeline. Any prediction with probability < 70% is silently discarded. This effectively rejects the confident-but-wrong majority-class predictions that occur when the user signs a minority letter.

3. Additional data collection sessions targeting the minority classes until all classes had ≥ 500 samples.

#### Challenge 5: TTS Blocking the Inference Loop

**Problem:** Calling `pyttsx3`'s `engine.say()` and `engine.runAndWait()` from the main inference thread blocked the video pipeline for the entire duration of speech, causing the camera feed to freeze while a word was being spoken.

**Solution:** The `TextToSpeech` class implements a Singleton pattern with a dedicated background daemon thread. Speech requests are placed on a `queue.Queue`. The worker thread dequeues items and calls the TTS engine sequentially, while the inference thread continues processing frames uninterrupted. The Singleton pattern ensures only one TTS engine instance exists across the entire application, preventing conflicts between the Tkinter dashboard and any Flask request handler that might both attempt to speak simultaneously.

#### Challenge 6: Word Boundary Detection Timing

**Problem:** Tuning the word boundary timeout was non-trivial. Too short (< 0.5s) caused false word breaks during brief hand repositioning between letters. Too long (> 1.5s) made the system feel sluggish and unresponsive.

**Solution:** The `HAND_LOST_GRACE_SEC = 0.5s` grace period was introduced to distinguish brief hand tracking drops (covered by the grace period) from genuine word pauses (which expire the grace period and then trigger the 0.8s boundary timer). Empirical testing across multiple signing sessions established that 0.5s grace + 0.8s boundary strikes the optimal balance between sensitivity and robustness for the target signing speed.

#### Challenge 7: All-Caps TTS Mispronunciation

**Problem:** The word assembler outputs words in uppercase (e.g., "HELLO"). When passed directly to `pyttsx3`, the TTS engine reads the word letter-by-letter ("H, E, L, L, O") rather than pronouncing it as a word.

**Solution:** The `TextToSpeech._normalize()` static method converts all-uppercase strings to Title Case before passing them to the engine (`"HELLO"` → `"Hello"`, `"HELLO WORLD"` → `"Hello World"`). Mixed-case strings (e.g., emergency phrases already in sentence case) pass through unchanged.

---

### 5.5 Code Quality Practices

**Docstrings:** All public methods are documented with parameter types, return types, and a brief description of behaviour.

**Configuration centralisation:** All numeric thresholds, colour values, file paths, and hardware parameters are defined exclusively in `config.py`. No magic numbers appear in the core pipeline modules.

**Graceful degradation:** Every component is wrapped in try/except blocks at the interface level. A failure in the TTS engine, camera, or classifier does not crash the application — it degrades gracefully with an appropriate error message.

**Module interfaces:** Each module exposes a minimal, stable public interface. Internal implementation details are communicated via name mangling (single underscore prefix), reducing accidental coupling.

**Singleton pattern for TTS:** Prevents multiple TTS engine instances from interfering with each other across the Tkinter and Flask interfaces, which both invoke TTS independently.

---

---

## CHAPTER SIX: TESTING AND EVALUATION

### 6.1 Introduction

System testing is the process by which a developed system is evaluated against its specified requirements to confirm that it behaves correctly, reliably, and efficiently. This chapter documents the testing strategy and results for BridgeSign, covering unit testing, integration testing, system acceptance testing, and model performance evaluation.

---

### 6.2 Testing Strategy

BridgeSign's testing approach is structured in four levels, following the standard V-model:

| Level | Scope | Tools / Approach |
|---|---|---|
| Unit Testing | Individual module behaviour | Python scripts, `_test_assembler.py`, `_test_tts_normalize.py`, `_check_accuracy.py` |
| Integration Testing | Module interaction | Combined pipeline execution with controlled inputs |
| System Testing | End-to-end user scenarios | Live signing sessions, manual acceptance checklist |
| Performance Testing | Frame rate, latency, memory | Time measurement, Windows Task Manager, profiling |

---

### 6.3 Unit Testing

#### 6.3.1 Word Assembler Unit Tests (`_test_assembler.py`)

The `_test_assembler.py` script tests the `WordAssembler` state machine with simulated letter sequences, without requiring a camera or model:

| Test Case | Input Sequence | Expected Output | Pass / Fail |
|---|---|---|---|
| Simple word | Push H, E, L, L, O → pause hand | Word = "HELLO" | ✅ Pass |
| Deduplication | Push H, E, L, L, L, L, O → pause | Word = "HELLO" (deduplicated) | ✅ Pass |
| Single letter | Push I → pause | Word = "I" | ✅ Pass |
| Known word match | Push W, O, R, L, D → pause | Word = "WORLD" (dictionary match) | ✅ Pass |
| Two-word sentence | Push H, E, Y → pause → push B, Y → long pause | Sentence = "HEY BY" | ✅ Pass |
| Buffer overflow | Push 23 letters without pause | Auto-flush at MAX_BUFFER_LEN | ✅ Pass |
| Manual flush | Push H, I → call manual_flush() | Word = "HI" immediately | ✅ Pass |
| Grace period | Push H → briefly remove hand (< 0.5s) → continue | No word break | ✅ Pass |

#### 6.3.2 TTS Normalisation Unit Tests (`_test_tts_normalize.py`)

| Input | Expected Output | Pass / Fail |
|---|---|---|
| `"HELLO"` | `"Hello"` | ✅ Pass |
| `"HELLO WORLD"` | `"Hello World"` | ✅ Pass |
| `"I need help."` | `"I need help."` (unchanged — mixed case) | ✅ Pass |
| `""` (empty) | `""` | ✅ Pass |
| `"A"` | `"A"` | ✅ Pass |

#### 6.3.3 Classifier Accuracy Check (`_check_accuracy.py`)

The `_check_accuracy.py` script loads the trained model and evaluates it against a held-out test subset, reporting per-class accuracy:

```
Overall accuracy: 91.4%

Per-class accuracy:
  A: 96.2%    B: 94.1%    C: 93.8%    D: 90.5%
  E: 88.2%    F: 91.0%    G: 84.7%    H: 87.3%
  I: 95.1%    J: 82.4%    K: 85.0%    L: 93.7%
  M: 83.2%    N: 81.5%    O: 92.6%    P: 86.1%
  Q: 83.8%    R: 89.4%    S: 88.7%    T: 84.3%
  U: 87.6%    V: 86.9%    W: 90.2%    X: 82.1%
  Y: 88.0%    Z: 79.3%
```

All letters achieve ≥ 79% accuracy, satisfying the minimum per-class accuracy requirement of 70% (NFR-3). The overall accuracy of 91.4% exceeds the 85% target (NFR-2).

---

### 6.4 Integration Testing

Integration testing verified correct data flow between adjacent modules:

| Integration Point | Test | Result |
|---|---|---|
| Camera → HandDetector | Camera produces valid BGR frames consumed by MediaPipe without shape errors | ✓ Pass |

| HandDetector → FeatureExtractor | Landmark list of 21 points produces 42-element normalised feature vector | ✓ Pass |

| FeatureExtractor → Classifier | Feature vector is accepted by `pipeline.predict_proba()` without dtype errors | ✓ Pass |

| Classifier → WordAssembler | Letter string from classifier is correctly appended by `push_letter()` | ✓ Pass |

| WordAssembler → TTS | Completed word from assembler is correctly passed to `speak_async()` | ✓ Pass |

| Flask API → Camera thread | Starting/stopping camera via API correctly controls the background thread | ✓ Pass |

| SessionTracker → JSON persistence | Each `log_translation()` call appends correctly to `data/sessions.json` | ✓ Pass |

---

### 6.5 System Acceptance Testing

System acceptance testing was performed through a structured live signing session evaluating all five functional tabs of the GUI.

#### 6.5.1 Live Camera Tab Acceptance

| Test | Expected Behaviour | Result |

|---|---|---|

| Start camera | Video feed opens; landmark overlay visible | ✓ Pass |

| Sign letter 'A' | Letter displayed in buffer; TTS vocalises "A" (or word) at boundary | ✓ Pass |

| Sign "HELLO" | H-E-L-L-O assembled; TTS speaks "Hello"; word appears in LAST WORD display | ✓ Pass |

| Sign "I AM FINE" | Three-word sentence assembled; TTS speaks "I Am Fine" at sentence completion | ✓ Pass |

| Unknown/noisy gesture | No letter added (confidence gate active); buffer remains empty | ✓ Pass |

| Next Word button | Immediately flushes buffer; TTS speaks partial word | ✓ Pass |

| Undo Letter button | Last letter removed from buffer; display updates | ✓ Pass |

| Clear All button | Buffer, word, sentence, and log cleared | ✓ Pass |

| Stop camera | Camera feed stops; resources released | ✓ Pass |

#### 6.5.2 Image Tab Acceptance

| Test | Expected | Result |

|---|---|---|

| Upload image with 'B' handshape | Displays "B" with confidence | ✓ Pass |

| Upload image with no hand | Displays "No hand detected" | ✓ Pass |

| Speak Result button | TTS speaks the detected label | ✓ Pass |

#### 6.5.3 Emergency Tab Acceptance

| Test | Expected | Result |

|---|---|---|

| Click "Speak" on "I need help." | Immediate TTS vocalisation | ✓ Pass |

| Add custom phrase | Phrase appears in grid | ✓ Pass |

| Delete custom phrase | Phrase removed from grid and storage | ✓ Pass |

#### 6.5.4 Stats Tab Acceptance

| Test | Expected | Result |

|---|---|---|

| After signing 5 words | Total translations count = 5 | ✓ Pass |

| Most common sign displayed | Correct letter/word shown | ✓ Pass |

| Refresh button | Stats update to reflect latest session | ✓ Pass |

---

### 6.6 Performance Testing

Performance benchmarks were measured on a development machine (Intel Core i5-11th Gen, 8GB RAM, integrated webcam):

| Metric | Target | Achieved |
|---|---|---|
| Video frame rate (camera tab) | ≥ 25 FPS | ~28–30 FPS |
| MediaPipe inference latency (per frame) | ≤ 33ms | ~18–25ms |
| Classifier prediction latency | ≤ 5ms | ~2–4ms |
| GUI update rate (Tkinter) | Capped at 30 FPS | 30 FPS (capped) |
| TTS response latency (word spoken after boundary) | ≤ 1.0s | ~0.3–0.8s |
| RAM usage (idle + camera running) | ≤ 1.5GB | ~680–920MB |
| CPU usage (single core, camera running) | ≤ 80% | ~45–65% |

All performance targets are satisfied. The system runs comfortably on the minimum hardware specification.

---

### 6.7 Known Limitations

Honest documentation of system limitations guides future work and sets appropriate expectations for users:

1. **Static signs only:** BridgeSign recognises only static handshape letters. Dynamic ASL signs (e.g., the full motion form of "HELLO", "THANK YOU", "LOVE") require temporal modelling not present in v1.0.

2. **Single hand only:** Only the dominant (first detected) hand is processed. Two-handed signs are not supported.

3. **Lighting sensitivity:** While the dataset was collected under varied conditions, extreme lighting (heavy backlighting, near-darkness) can degrade detection confidence.

4. **'J' and 'Z' limitation:** ASL 'J' and 'Z' both involve motion (a curved J-shape and a Z-stroke respectively). The static model recognises their starting or ending postures, which may not always be distinct from 'I' or 'S'.

5. **Webcam quality dependency:** Very low-resolution cameras (< 480p) or cameras with significant motion blur may produce degraded landmark quality.

---

---

## CHAPTER SEVEN: CONCLUSION AND FUTURE WORK

### 7.1 Conclusion

This project set out to design, implement, and evaluate BridgeSign — a real-time American Sign Language recognition and translation system that bridges the communication gap between Deaf signers and hearing non-signers using only a standard webcam and a personal computer. The system was successfully implemented and evaluated, meeting or exceeding all specified functional requirements and non-functional performance targets.

The core contributions of BridgeSign may be summarised as follows:

**Architectural contribution:** A fully modular, layered Python architecture separating camera capture, hand detection, feature extraction, classification, word assembly, TTS output, and GUI presentation into independently testable and replaceable components. This architecture achieves all major software engineering quality attributes: maintainability, extensibility, reliability, and portability.

**Technical contribution:** An end-to-end real-time ASL recognition pipeline operating at 30 FPS on commodity CPU hardware, combining Google's MediaPipe Hand Landmarker for pose estimation with a

soft-voting ensemble of Random Forest and SVM classifiers for sign classification, achieving 91.4% overall accuracy and ≥ 79% per-class accuracy across all 26 letters.

**Temporal modelling contribution:** A novel WordAssembler state machine implementing consecutive-frame stability gating, confidence-threshold-based rejection, hand-loss grace periods, dictionary-validated word assembly, and sentence boundary detection — converting noisy per-frame letter predictions into correctly bounded, naturally spoken words and sentences.

**Accessibility contribution:** An inclusive multi-interface design (Tkinter desktop + Flask web) featuring a dedicated Emergency Phrases module for crisis communication, a Learning Mode for sign language education, and a Statistics dashboard for session monitoring — all accessible offline and free of charge.

BridgeSign demonstrates that a resource-efficient, thoughtfully engineered approach combining established open-source tools can produce a practically useful accessibility aid without requiring cloud infrastructure, specialised hardware, or institutional resources. It is hoped that this work serves both as a deployable practical tool and as a reproducible academic template for future SLR research.

---

### 7.2 Future Work

The following extensions are recommended for future versions of BridgeSign:

#### 7.2.1 Dynamic Sign Recognition

The most impactful extension to BridgeSign would be the addition of dynamic (motion-based) sign recognition. ASL employs many high-frequency signs (HELLO, THANK YOU, YES, NO, PLEASE, SORRY) that are expressed as movements rather than static handshapes. A temporal classifier — such as an LSTM, Temporal Convolutional Network, or Transformer — operating on rolling windows of hand landmark trajectories would enable recognition of these signs without requiring users to finger-spell every word.

#### 7.2.2 Full ASL Vocabulary Support

Beyond the finger-spelled alphabet, a comprehensive dictionary of ASL word signs could be added. This would shift the interaction model from character-level composition to direct word recognition — significantly increasing communication speed for fluent signers.

#### 7.2.3 Bidirectional Communication

Future versions could incorporate speech recognition (STT) to allow the hearing conversation partner to speak into a microphone, producing displayed text for the Deaf user to read — creating a fully bidirectional communication tool within a single application.

#### 7.2.4 Mobile Application

A mobile companion app for Android and iOS would dramatically expand accessibility, allowing Deaf individuals to use the tool in public settings without requiring a laptop and webcam. This could be implemented using TensorFlow Lite (for on-device inference) or by connecting to a hosted Flask API.

#### 7.2.5 Multi-Language Sign Support

Extending BridgeSign to support additional sign languages — British Sign Language (BSL), Indian Sign Language (ISL), Nigerian Sign Language (NSL) — would require new training datasets for each language but no architectural changes. A language selector in the GUI would allow users to switch between language-specific classifier models.

#### 7.2.6 Continuous Learning from User Feedback

A system that allows users to provide correction feedback ("this was recognised incorrectly") and uses that feedback to incrementally retrain the model would create a continuously improving personalised recognition system, adapting to the specific signing style of each user.

#### 7.2.7 Confusion Pair Analysis and Specialised Training

The confusion matrix from model evaluation reveals specific letter pairs that are frequently confused (e.g., M/N, G/H, S/T). Future work could apply targeted data augmentation — generating synthetic

variations of these specific signs — to sharpen decision boundaries between the most commonly confused pairs.

#### 7.2.8 Haptic Feedback Integration

For Deaf-Blind users, a companion haptic feedback device (such as a vibrating wristband or Braille display) could receive the recognised word output, extending the system's accessibility beyond the visual modality.

---

### 7.3 Final Remarks

The development of BridgeSign represents a genuine effort to apply modern software engineering and machine learning techniques in service of social equity and accessibility. Communication is foundational to human dignity; tools that expand the circle of who can communicate — and how — contribute meaningfully to the broadening of that dignity. It is the sincere hope of the author that BridgeSign, however modest its scope, makes at least one conversation possible that would not have happened otherwise.

---

---

## REFERENCES

Bradski, G. R. (1998). *Computer vision face tracking for use in a perceptual user interface*. Intel Technology Journal, Q2, 1–15.

Cheok, M. J., Omar, Z., Jaward, M. H. (2019). A review of hand gesture and sign language recognition techniques. *International Journal of Machine Learning and Cybernetics*, 10(1), 131–153. https://doi.org/10.1007/s13042-017-0705-5

Chollet, F. (2017). *Deep learning with Python*. Manning Publications.

Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

Koller, O., Forster, J., & Ney, H. (2015). Continuous sign language recognition: Towards large vocabulary statistical recognition systems handling multiple signers. *Computer Vision and Image Understanding*, 141, 108–125.

Mace, R. L., Hardie, G. J., & Place, J. P. (1991). *Accessible environments: Toward universal design*. Design for Independent Living.

OpenCV. (2024). *Open source computer vision library*. https://opencv.org/

Pigou, L., Dieleman, S., Kindermans, P. J., & Schrauwen, B. (2014). Sign language recognition using convolutional neural networks. *ECCV 2014 Workshop on Assistive Computer Vision and Robotics*.

Rao, G. A., Kishore, P. V. V., & Kumar, E. K. (2018). Sign language recognition system using HOG and SVM. *International Journal of Engineering & Technology*, 7(4), 4874–4879.

Scikit-learn Developers. (2024). *scikit-learn: Machine learning in Python*. https://scikit-learn.org/

Story, M. F., Mueller, J. L., & Mace, R. L. (1998). *The universal design file: Designing for people of all ages and abilities*. NC State University, Center for Universal Design.

Wadhawan, A., & Kumar, P. (2021). Sign language recognition systems: A decade systematic literature review. *Archives of Computational Methods in Engineering*, 28(3), 785–813. https://doi.org/10.1007/s11831-019-09384-2

World Health Organization. (2023). *Deafness and hearing loss — Fact sheet*. https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C. L., & Grundmann, M. (2020). MediaPipe hands: On-device real-time hand landmark detection. *arXiv preprint arXiv:2006.10214*. https://arxiv.org/abs/2006.10214

---

---

*End of BridgeSign Academic Documentation — Version 1.0*

*Total Chapters: 7 | Estimated Print Length: 40–50 pages*

---

> **Final Note to Student:** Replace all `[Your Name]`, `[Your Institution Name]`, `[Department Name]`, and `[Supervisor Name]` placeholders on the title page before submission. Insert actual screenshots of the running application at every figure reference. Replace the sample accuracy table values in Section 6.3.3 with your own actual model output from `sign_model_metrics.json`. Page numbers in the List of Figures should be updated once the document is typeset in Word or LaTeX.

---

That is the complete documentation you originally posted. Let me know if you need any specific section highlighted, explained, or revised!

i want you to write me a proper documenetaion ggeneret a wrd document so that i can just download it i need a few adjustment ments on the structure so you start with abstruct then acknolodgements you family suppervisor what what god then table of contents on digrams chapter 3 after systems design,introduction system archoiture the first digram will be the case digram data flow digram where we have three digrams there for level 1,2,3 then the ctivity digram then the compenetn the sequence then entity realtionship so just leave space for me to paste this daigrams and also for the screns others and list of whatw ht where we state the figure edit it and there shud be no things that are not with generete document ooooh and i shud not be caught that i used ai no